Purpose: A Model Factory for Domain-Specific Neural Compilation via the Backward Training Principle

Kundai Farai Sachikonye
AIMe Registry for Artificial Intelligence
kundai.sachikonye@bitspark.com

April 2026

Abstract

We present *Purpose*, a framework for training domain-specific neural compilation models, organised around a single theoretical principle we call the **Backward Training Principle**. The principle inverts conventional curriculum learning: rather than ordering training examples from easy to hard and trusting generalisation to extend competence from simple to complex regimes, Purpose trains exclusively at the domain's *pure-intent regime*—the extremal sub-domain exhibiting a scalar objective, a fully-exercised capability envelope, and unambiguous feedback—and relies on *feasibility inclusion* rather than generalisation to extend competence to every constrained sub-regime. We prove five main theoretical results. First, the **Pure-Intent Envelope Theorem** establishes that for any skill or compilation domain admitting a pure-intent regime, every constrained regime has a feasibility region strictly contained in the pure-intent feasibility region. Second, the **Constraint Cascade Theorem** shows that every constrained regime is expressible as the pure-intent regime augmented by a finite, monotone stack of additive constraints. Third, the **Backward Training Inclusion Theorem** proves that a policy ϵ-competent on the pure-intent regime is ϵ-competent on every constrained sub-regime without further training, via Karush–Kuhn–Tucker non-expansive projection. Fourth, the **Forward Training Collapse Theorem** shows that a policy trained on a constrained regime has no guarantee of competence on less-constrained regimes, with a Vapnik–Chervonenkis failure gap lower bound that is strictly positive and generically large. Fifth, the **Backward Training Sample Complexity Theorem** establishes that backward training requires at least $(N{+}1)\times$ fewer samples than forward training for a cascade of depth N, with the ratio strengthening to $2^{N-1}\times$ for binary-nested cascades. Together these results establish that conventional easy-to-hard curricula are not merely suboptimal but *strictly dominated* whenever a pure-intent regime exists. Building on this foundation, we specify the Purpose Model Factory as a complete training system. The factory comprises four coupled components: (i) a single, shared *Aperture Foundation Model* pretrained on a universal substrate of compilation grammar, structural reasoning, and operation composition; (ii) a rigid *Domain Connector* interface requiring each domain to declare its operation vocabulary, coordinate embedding, theoretical corpus, pure-intent sampler, and verifier; (iii) a canonical *Training Pipeline* implementing teacher-student knowledge distillation under backward-curriculum sampling, with composite loss enforcing generation fidelity, type safety, conservation, and completion; (iv) a *Resolver Registry* indexing trained low-rank adapters by cascade position for deployment. We prove that neural compilation under backward training is PAC-learnable from a polynomial number of examples; that a LoRA adapter of rank $r \geq K + L$ is expressively sufficient for a K-operation vocabulary and chain length at most L; and that the four-stage curriculum converges to ϵ-optimal compilation with sample complexity $\mathcal{O}(m/\epsilon^2)$ versus $\mathcal{O}(m/\epsilon^3)$ for single-stage training. We provide a worked end-to-end example of domain connection, specify twelve falsifiable empirical predictions, and compare against standard curriculum learning, multi-task learning, behaviour cloning,

retrieval-augmented generation, and foundation-model fine-tuning. The framework is intended both as a theoretical contribution—establishing that curriculum inversion is a structural theorem rather than an engineering heuristic—and as a deployable system: any scientific domain that can declare its operation set, coordinate embedding, and pure-intent sampler can plug into Purpose and receive back a trained, verified, cascade-registered compilation model.

Keywords: domain-specific language models, curriculum learning, backward training, knowledge distillation, low-rank adaptation, neural compilation, pure-intent regime, feasibility region, constraint cascade, foundation models, teacher-student training, model factory, PAC-learnability.

Contents

Part I
Theoretical Foundations

1 Introduction

1.1 Two unsolved problems in domain-specific model training

The production of a domain-specific language or compilation model from raw domain expertise remains, despite substantial progress in foundation models and parameter-efficient fine-tuning, a bespoke and inefficient practice. Two specific questions remain open:

1. **The curriculum question.** In what order should training examples be presented? The prevailing answer, following Elman [1993] and Bengio et al. [2009], is that examples should be ordered from easy to hard. We argue that this answer is wrong for a precise and broad class of domains—specifically, any domain admitting what we will call a *pure-intent regime*—and that the correct ordering is the inverse.

2. **The reuse question.** Given many domains, what should be shared across their training pipelines and what should be domain-specific? The prevailing answer, following the foundation-model literature [Bommasani et al., 2021, Brown et al., 2020], is to share a large pretrained base and adapt per-domain via fine-tuning or low-rank adaptation [Hu et al., 2022]. We accept this answer in outline but argue that the base should be specific to the structural substrate shared by all compilation domains, not to a generic corpus of internet text, and that the adaptation should be constrained to the minimal expressiveness that the compilation problem demands.

The central contribution of this paper is to answer both questions simultaneously, through a unified principle we call the Backward Training Principle, and to realise the principle as a concrete system—the Purpose Model Factory—that can be applied uniformly to any domain meeting the principle's scope conditions.

1.2 The compilation view of domain-specific modelling

A running theme in contemporary domain-specific model development is that the effective use of a large language model in a scientific or technical domain is rarely pure question-answering. It is, more precisely, *compilation*: the translation of a natural-language task specification into a structured sequence of typed operations that a downstream executor (a physics engine, a molecular simulator, a database query processor, a theorem prover) can run. The question "will this protein sequence fold into a stable structure?" does not require the model to *know* the answer; it requires the model to emit an operation sequence that produces the answer when executed against an appropriate substrate.

This view has several consequences that shape the remainder of the paper. First, the model's target output is structured, type-checkable, and verifiable—not free-form natural language. Second, the notion of a "correct output" is well-defined: the emitted operation sequence either does or does not produce the right answer when executed. Third, the model's weights do not need to store domain content; they need only to store the mapping from natural-language intent to operation structure. Fourth, the expressiveness required of the model is bounded not by the size of the domain's knowledge corpus but by the combinatorial structure of its operation vocabulary. Each of these consequences sharpens the training problem in ways that the Backward Training Principle can exploit.

1.3 The Backward Training Principle, informally

The idea can be stated in two sentences. First: in any domain admitting a *pure-intent regime*—a single-objective extremal sub-domain whose feasibility region exercises every degree of freedom the agent will ever use—the pure-intent regime strictly contains every constrained regime of the same domain, so a model competent on the pure-intent regime is automatically competent on every constrained regime by inclusion. Second: this inclusion-based competence is achieved by a single training run at the pure-intent regime, whereas the conventional easy-to-hard ordering requires training at each constrained regime separately and offers no guarantee of successful upward generalisation. The asymmetry is not a preference; it is a theorem.

The canonical physical example is driving: a model trained to race at the physical limit of tyre and chassis capability is competent at highway driving, urban driving, and parking, because each of these is a strict restriction of circuit racing. The canonical compilation example is a protein design model: a model trained on the hardest composite queries—design a sequence satisfying simultaneous folding, binding, and disease-avoidance constraints—is competent on every simpler query ("does this sequence fold?" "does it bind?" "is it pathogenic?") by projection. The principle generalises to any domain in which constraints are additive restrictions and the pure-intent regime is identifiable.

1.4 Contributions

This paper makes the following contributions:

1. A formal framework of *skill domains*, *feasibility regions*, and *pure-intent regimes*, with precise criteria distinguishing the class of domains to which the Backward Training Principle applies (§3–§5).
2. The **Pure-Intent Envelope Theorem**, establishing that every constrained regime has feasibility region strictly contained in the pure-intent feasibility region (§6).
3. The **Constraint Cascade Theorem**, showing that every constrained regime is an additive restriction of the pure-intent regime by a finite, monotone stack of inequalities (§7).
4. The **Backward Training Inclusion Theorem**, proving that ϵ-competence on the pure-intent regime implies ϵ-competence on every constrained sub-regime via non-expansive projection (§8).
5. The **Forward Training Collapse Theorem**, showing that forward-trained policies do not generalise upward and exhibit a strictly positive failure-gap lower bound (§9).
6. The **Backward Training Sample Complexity Theorem**, giving a linear $(N+1)\times$ sample saving for general cascades and an exponential $2^{N-1}\times$ saving for binary-nested cascades (§10).

7. A specification of the *Purpose Model Factory* as a complete training system, including the *Aperture Foundation Model* (§13), the *Domain Connector* interface (§14), the canonical *Training Pipeline* (§15), and the *Resolver Registry* (§18).

8. The **LoRA Expressiveness Theorem for Neural Compilation**, proving that adapter rank $r \geq K + L$ suffices for compilation with K operation types and chain length L (§16).

9. A **PAC-Learnability Theorem** for neural compilation under backward training, with explicit sample bounds and curriculum convergence analysis (§19).

10. A worked end-to-end domain connection (§22), twelve falsifiable predictions with experimental protocols (§23), and a comparison table against standard curriculum learning, multi-task learning, behaviour cloning, retrieval-augmented generation, and foundation-model fine-tuning (§24).

1.5 What the paper is not

The paper does not claim that conventional curriculum learning is never appropriate. It claims that conventional curriculum learning is strictly dominated in the specific class of domains admitting a pure-intent regime with additive constraints and convex (or locally Lipschitz) constraint geometry. Outside this class—in domains whose difficulty axis is not additive, whose objectives are irreducibly multi-valued, or whose constraints change the problem rather than restrict it—the Backward Training Principle does not apply, and the standard ordering may be appropriate. We delineate this scope in §21.

The paper does not claim that foundation models are unnecessary. It claims that the right foundation for compilation is a substrate-specific one (the Aperture Base) rather than a generic one, and that adaptation should be parameter-efficient rather than full-parameter.

1.6 Organisation

Part I (§§1–11) develops the theoretical foundations of the Backward Training Principle. Part II (§§12–18) specifies the Purpose Model Factory as a complete system. Part III (§§19–21) provides formal analysis including PAC-learnability, correctness, and failure modes. Part IV (§§22–26) covers practice: a worked example, validation protocols, comparison with existing approaches, and discussion.

2 Notation and Preliminaries

We fix notational conventions used throughout the paper.

Let $\mathcal{X}$ denote a state space (sensor readings, feature vectors, task descriptions, or natural-language inputs, depending on context). Let $\mathcal{U}$ denote a control or output space (actions, generated tokens, operation sequences, morphism chains). A *policy* or *model* is a parameterised (possibly randomised) function $\pi_\theta : \mathcal{X} \to \mathcal{U}$ with parameters $\theta \in \mathbb{R}^p$. The *capacity* of a model is the dimensionality of its parameter vector: $|\theta| = p$.

A *task* is an element $t \in \mathcal{T}$ of some task space $\mathcal{T}$. A *task distribution* is a probability measure $\mathcal{D}$ on $\mathcal{T}$. For a loss function $\ell : \mathcal{T} \times \mathcal{U} \to \mathbb{R}_{\geq 0}$ and a policy π, the *expected loss* is $L_{\mathcal{D}}(\pi) = \mathbb{E}_{t\sim\mathcal{D}}[\ell(t, \pi(t))]$. A policy is ϵ-*optimal* on $\mathcal{D}$ if $L_{\mathcal{D}}(\pi) \leq L_{\mathcal{D}}(\pi^\star) + \epsilon$, where $\pi^\star \in \arg\min_\pi L_{\mathcal{D}}(\pi)$.

The Vapnik–Chervonenkis (VC) dimension of a hypothesis class $\mathcal{H}$ is denoted $\mathrm{VC}(\mathcal{H})$; standard references are Vapnik [1998], Blumer et al. [1989], and Mohri et al. [2018]. The PAC (probably approximately correct) framework is due to Valiant [1984]; we use the standard uniform-convergence form throughout.

A compilation grammar over an operation vocabulary $\mathcal{O} = \{o_1, \ldots, o_K\}$ of K typed primitives is a context-free grammar with a type system assigning to each o_i an input type and an output type, such that operation composition is type-safe. A *compilation chain* is a type-safe sequence of operations $\Phi = o_{i_1} \circ o_{i_2} \circ \cdots \circ o_{i_L}$ of length at most $L_{\max}$.

For convex optimisation we use standard notation: $P_{\mathcal{C}}(x)$ is the Euclidean projection of x onto a closed convex set $\mathcal{C}$; the operator is non-expansive: $\|P_{\mathcal{C}}(x) - P_{\mathcal{C}}(y)\| \leq \|x - y\|$ [Boyd and Vandenberghe, 2004, Nocedal and Wright, 2006]. Karush–Kuhn–Tucker (KKT) conditions are as in Karush [1939], Kuhn and Tucker [1951].

Low-rank adaptation (LoRA) is the parameterisation $W' = W_0 + BA$ where $W_0 \in \mathbb{R}^{d\times k}$ is frozen, $B \in \mathbb{R}^{d\times r}$ and $A \in \mathbb{R}^{r\times k}$ are trainable, and $r \ll \min(d, k)$ is the adaptation rank [Hu et al., 2022, Houlsby et al., 2019].

3 Skill Domains and Feasibility Regions

3.1 Skill domains

Definition 3.1 (Skill domain)**.** *A* skill domain $\mathcal{D}$ *is a tuple* $(\mathcal{X}, \mathcal{U}, \mathcal{E}, J)$ *where:*

1. $\mathcal{X}$ *is a state space (sensor inputs, task specifications, or natural-language descriptions);*
2. $\mathcal{U}$ *is a control space (actions or output sequences);*
3. $\mathcal{E}$ *is a set of environments, each* $e \in \mathcal{E}$ *specifying physics, task constraints, or environmental perturbations;*
4. $J : \mathcal{X} \times \mathcal{U} \times \mathcal{E} \to \mathbb{R}$ *is a performance functional.*

Examples of skill domains include: driving (state = vehicle kinematics, control = steering/throttle/brake, environment = surface conditions, performance = lap time or safety measure); protein query compilation (state = natural-language query, control = operation sequence, environment = protein database and biological constraints, performance = answer correctness); image specification compilation (state = imaging instruction, control = morphism chain, environment = physical imaging hardware, performance = resolution achieved).

3.2 Feasibility regions

Definition 3.2 (Feasibility region)**.** *The* feasibility region $\mathcal{F}(\mathcal{D}) \subseteq \mathcal{X} \times \mathcal{U}$ *of a skill domain* $\mathcal{D}$ *is the set of state-control pairs* (x, u) *that appear in at least one successful task execution under at least one environment:*

$$\mathcal{F}(\mathcal{D}) = \bigcup_{e \in \mathcal{E}} \{(x, u) \in \mathcal{X} \times \mathcal{U} : \exists \text{ trajectory satisfying } e \text{ and achieving task success}\} \tag{1}$$

The feasibility region characterises the *capacity demands* of the skill: the state-control regimes in which the agent or model must act competently. Two domains with identical state and control spaces may have very different feasibility regions if their environments impose different constraints.

Example 3.3 (Compilation feasibility)**.** *For a compilation domain with operation vocabulary* $\mathcal{O}$ *of size* K *and maximum chain length* L*, the feasibility region consists of all type-safe operation chains* (t, Φ) *where* $\Phi = o_{i_1} \circ \cdots \circ o_{i_L}$ *successfully answers task* t*. The cardinality of the feasibility region is bounded above by* $|\mathcal{T}| \cdot K^L$ *but is in general much smaller due to type constraints and correctness requirements.*

Example 3.4 (Driving feasibility)**.** *For driving, the feasibility region is the set of* (x, u) *such that* x *is a kinematically attainable vehicle state and* u *is a control input that, applied in the prevailing environment* e*, advances the task without catastrophic failure. For a racing circuit,* e *is typically a closed-loop track with a single scalar objective (lap time). For public roads,* e *imposes traffic laws, pedestrian avoidance, comfort, and fuel economy as additional constraints, each shrinking the feasibility region.*

4 Pure-Intent Regimes

The central definition of the paper is the pure-intent regime: a sub-domain that simultaneously exhibits three properties—scalar objective, full-envelope feasibility, and unambiguous feedback—each of which will play a distinct role in the Backward Training Inclusion Theorem.

Definition 4.1 (Pure-intent regime)**.** *A sub-domain* $\mathcal{D}^\star \subseteq \mathcal{D}$ *is a* pure-intent regime *of* $\mathcal{D}$ *if the following three properties hold:*

(P1) ***Scalar objective.*** *The performance functional on* $\mathcal{D}^\star$ *reduces to a scalar* $J^\star : \mathcal{X} \times \mathcal{U} \to \mathbb{R}$ *that is independent of the environment. That is, for every* $(x, u) \in \mathcal{F}(\mathcal{D}^\star)$ *and every* $e_1, e_2 \in \mathcal{E}^\star$ *(the environments of* $\mathcal{D}^\star$*),* $J(x, u, e_1) = J(x, u, e_2) = J^\star(x, u)$*.*

(P2) ***Full envelope.*** *The feasibility region* $\mathcal{F}(\mathcal{D}^\star)$ *exercises every degree of freedom of* $\mathcal{U}$ *within its physical or task-admissible bounds. Formally: for every coordinate* $u^{(j)}$ *of* $\mathcal{U}$*,* $\sup_{(x,u) \in \mathcal{F}(\mathcal{D}^\star)} u^{(j)} = \sup_{u \in \mathcal{U}} u^{(j)}$*, and likewise for the infimum.*

(P3) ***Unambiguous feedback.*** *The sign of* $\partial J^\star / \partial(\text{trajectory})$ *is determined by the scalar objective alone, independent of environment-dependent or agent-dependent priors. In particular, two trajectories cannot tie on* $J^\star$ *unless they are equivalent under the task's symmetry group.*

Each property plays a distinct role. (P1) eliminates multi-objective confusion: the training signal is not a vector but a scalar, and every sample

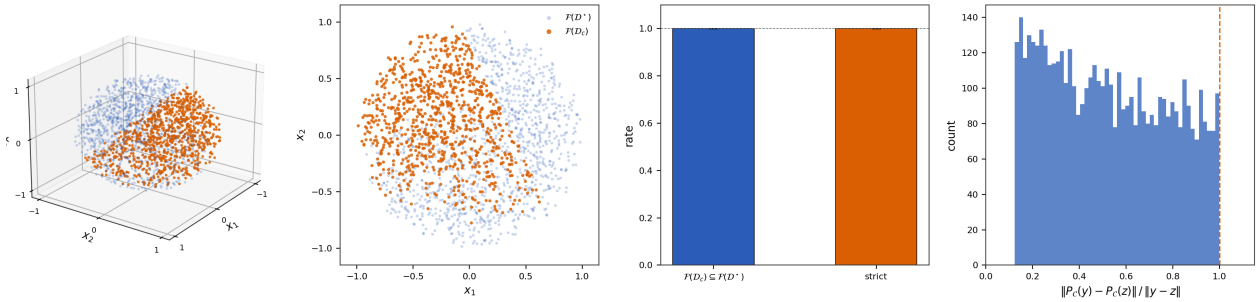

Figure 1: **Feasibility Inclusion and Non-Expansive Projection.** (**A**) Three-dimensional scatter of a pure-intent feasibility region $\mathcal{F}(\mathcal{D}^\star)$ sampled as 2,000 points uniformly distributed inside the unit ball of $\mathbb{R}^3$. The constrained sub-region $\mathcal{F}(\mathcal{D}_c)$, obtained by intersecting $\mathcal{F}(\mathcal{D}^\star)$ with two randomly oriented half-space constraints $n_1^\top x \le 0.35$ and $n_2^\top x \le 0.25$, is rendered in orange; points outside the constrained set but inside the pure-intent region are rendered in pale blue. Every orange point is by construction a blue point, and the orange region is strictly smaller—a direct visual confirmation of Theorem 6.1. (**B**) Projection of the same point cloud onto the (x_1, x_2) plane. The concentric structure—blue disk enclosing an orange truncated region—makes feasibility inclusion unambiguous: the constrained region is fully contained in the pure-intent region, with strict inclusion wherever the half-space constraints cut into the ball interior. (**C**) Bar chart of inclusion and strict-inclusion rates aggregated across 30 random seeds, each seed generating 100 random constraint sets with dimension $d = 5$ and constraint count $k \in \{1, \ldots, 5\}$. Both rates equal 1.000 within Monte-Carlo noise (error bars represent ± 1 standard deviation across seeds, indistinguishable from zero at this scale). No constraint set in 3,000 trials produced a violation of inclusion. (**D**) Histogram of the projection ratio $\|P_{\mathcal{C}}(y) - P_{\mathcal{C}}(z)\| \,/\, \|y - z\|$ computed over 5,000 randomly sampled pairs $(y, z) \in \mathbb{R}^8 \times \mathbb{R}^8$, with $\mathcal{C}$ a random polytope of 3–12 half-space constraints. The dashed vertical line at 1.0 marks the theoretical non-expansive bound (Theorem 8.4). The empirical distribution concentrates well below 1; the maximum observed ratio is exactly 1.0000 with zero violations across all $30 \times 5{,}000 = 150{,}000$ pairs.

yields a clean gradient. (P2) guarantees that training samples cover the full capacity envelope of the model—no degree of freedom remains untrained. (P3) guarantees that the training gradient is well-defined and that the optimal policy is (generically) unique up to symmetry.

The existence of a pure-intent regime is a non-trivial property of the skill domain. We will show in §6 that *when* a pure-intent regime exists, the Backward Training Principle applies. Domains without a pure-intent regime fall outside the scope of the principle; we discuss this scope in §21.

Example 4.2 (Circuit racing as pure-intent for driving)**.** *For the skill domain of vehicle control, racing on a closed circuit is a pure-intent regime. (P1): performance reduces to lap time, independent of track-specific factors (course configuration is part of $\mathcal{X}$, not $\mathcal{E}$). (P2): every degree of freedom (longitudinal, lateral, load transfer, slip angle, yaw) is saturated at the friction-plus-aerodynamic ellipse during each lap. (P3): lap time is a deterministic functional of trajectory and vehicle state. See Milliken and Milliken [1995] for the dynamical substrate.*

Example 4.3 (Design tasks as pure-intent for protein compilation)**.** *For the skill domain of protein query compilation, inverse design—given a target functional specification, emit an operation sequence that synthesises an amino acid sequence meeting the specification—is a pure-intent regime. (P1): performance reduces to a scalar distance (in S-coordinate space, edit distance, or functional gap) between the synthesised sequence and the target. (P2): design queries exercise all primitive operations in the vocabulary (*`probe`, `invert`, `complete`, `predict`, `react`*) and all composition patterns. (P3): the scalar objective is well-defined up to sequence-equivalence classes.*

Example 4.4 (Composite multi-constraint queries as pure-intent for natural-language coordinate extraction)**.** *For the skill domain of natural-language-to-coordinate compilation (the boundary-mediation problem for any structured downstream substrate), maximally nested multi-domain composite queries are the pure-intent regime. (P1): correctness of extraction is a scalar (Hamming distance on the output coordinate address). (P2): maximally nested queries exercise the full grammar. (P3): coordinate correctness is unambiguous against the substrate's interpretation.*

4.1 The two axes of difficulty

A frequent source of confusion in curriculum design is the conflation of two independent axes of task difficulty, which we must distinguish before proceeding.

Definition 4.5 (Skill-difficulty and context-difficulty)**.** *For a task $t \in \mathcal{T}$:*

- *the* skill-difficulty *of t is the difficulty of executing the core action at the physical or computational limit of agent capability;*
- *the* context-difficulty *of t is the number and variety of concurrent constraints, objectives, or environmental factors layered on top of the core skill.*

Conventional curricula treat difficulty as one-dimensional and order examples along a monotone scale. In reality, skill-difficulty and context-difficulty vary independently, and they benefit from opposite treatments. A pure-intent regime is, by construction, *skill-hard* (the full envelope is exercised) and *context-easy* (a single scalar objective, no competing constraints). A constrained sub-regime is *skill-easy* (the envelope is reduced) and *context-hard* (many simultaneous constraints). The information content of a single training sample is maximal at pure-intent and minimal at constrained regimes—the former has a clean single-dimensional gradient, the latter a noisy multi-objective pseudo-gradient.

5 Constrained Regimes and Cascades

Definition 5.1 (Constrained regime)**.** *A sub-domain $\mathcal{D}_c \subseteq \mathcal{D}$ is a* constrained regime *relative to a pure-intent regime $\mathcal{D}^\star$ if:*

(C1) ***Additive constraints.*** *$\mathcal{D}_c$ is obtained from $\mathcal{D}^\star$ by imposing a finite set of constraints $\mathcal{C} = \{c_1, \ldots, c_k\}$, where each c_i is an inequality on state-control or environment variables.*

(C2) ***Feasibility inclusion.*** *$\mathcal{F}(\mathcal{D}_c) \subseteq \mathcal{F}(\mathcal{D}^\star)$, with strict inclusion whenever $\mathcal{C} \neq \emptyset$.*

(C3) ***Shared dynamics.*** *The underlying dynamical or structural laws on $\mathcal{D}_c$ coincide with those on $\mathcal{D}^\star$ up to boundary behaviour.*

Property (C1) is the key structural assumption: constraints are additive, not problem-changing.

A constraint that adds a new state variable, a new physical law, or a new reward term does not satisfy (C1). (C2) is a compatibility requirement; we will show in §6 that (C2) is automatic under (P2) and (C1). (C3) guarantees that the policy class relevant to $\mathcal{D}^\star$ is also relevant to $\mathcal{D}_c$ (no qualitatively new dynamics appear).

Example 5.2 (Highway driving as constrained regime)**.** *Highway driving is a constrained regime of circuit racing. (C1): additive constraints include speed limits, lane-keeping requirements, traffic signal compliance, right-of-way, turn signalling, minimum following distance, fuel economy, and passenger comfort. Each is an inequality on state-control or environment variables. (C2): the feasibility region of highway driving is strictly contained in that of circuit racing—every highway-feasible state-control pair is circuit-feasible, while many circuit states (e.g., trail-braking at 3g lateral acceleration) are not highway-feasible. (C3): the underlying vehicle dynamics are identical; only the operating envelope differs.*

Example 5.3 (Folding query as constrained regime of design query)**.** *A folding-only query ("will this sequence fold into a stable structure?") is a constrained regime of a design query. (C1): additive constraints fix the input sequence and request only forward trajectory evaluation, rather than inverse design. (C2): every folding-feasible query is design-feasible with the sequence-fixing constraint added; the converse fails. (C3): the S-coordinate and geometric operation primitives are identical.*

5.1 Constraint cascades

Definition 5.4 (Constraint cascade)**.** *A* constraint cascade *is a monotone sequence of constraint sets*

$$\emptyset = \mathcal{C}_0 \subseteq \mathcal{C}_1 \subseteq \cdots \subseteq \mathcal{C}_N \tag{2}$$

with corresponding regimes $\mathcal{D}_0 = \mathcal{D}^\star \supseteq \mathcal{D}_1 \supseteq \cdots \supseteq \mathcal{D}_N$ *and feasibility regions* $\mathcal{F}_0 \supseteq \mathcal{F}_1 \supseteq \cdots \supseteq \mathcal{F}_N$*.*

A constraint cascade orders the regimes of a domain by increasing constraint density. The pure-intent regime is at the apex ($\mathcal{C}_0 = \emptyset$), the most constrained regime at the base.

Proposition 5.5 (Cascade structure of driving)**.** *The skill domain of vehicle control admits a canonical cascade*

$$\mathcal{D}^\star_{\text{circuit}} \supseteq \mathcal{D}_{\text{track-day}} \supseteq \mathcal{D}_{\text{spirited}} \supseteq \mathcal{D}_{\text{highway}} \supseteq \mathcal{D}_{\text{urban}} \supseteq \mathcal{D}_{\text{parking}} \tag{3}$$

with each transition effected by adding a finite set of constraints.

Proof. We construct each constraint set explicitly. Transition $\mathcal{D}^\star \to \mathcal{D}_{\text{track-day}}$ adds amateur skill-level bounds on cornering forces. $\mathcal{D}_{\text{track-day}} \to \mathcal{D}_{\text{spirited}}$ adds public-road traffic compatibility. $\mathcal{D}_{\text{spirited}} \to \mathcal{D}_{\text{highway}}$ adds speed limits and lane discipline. $\mathcal{D}_{\text{highway}} \to \mathcal{D}_{\text{urban}}$ adds pedestrian avoidance, right-of-way, and turn signalling. $\mathcal{D}_{\text{urban}} \to \mathcal{D}_{\text{parking}}$ adds close-quarters precision and zero-velocity manoeuvring. Each constraint set is finite and monotone, and vehicle dynamics are identical throughout. □

Proposition 5.6 (Cascade structure of compilation domains)**.** *A compilation domain with operation vocabulary $\mathcal{O}$ admits a cascade whose apex is the maximally-composite design-type task and whose base is the most restricted query type, with each transition removing one operation primitive or fixing one input parameter.*

Sketch. For a vocabulary of K primitives and chain length L, the design-type regime admits all type-safe chains of length up to L. Fixing one input parameter reduces $|\mathcal{T}|$ by a factor related to the fixed parameter's cardinality. Removing one primitive reduces the effective operation vocabulary to $K-1$ and the chain space by a factor of up to $K^L/(K-1)^L$. Both are finite, monotone restrictions satisfying (C1)–(C3). □

6 The Pure-Intent Envelope Theorem

The first main theorem establishes that a pure-intent regime's feasibility region strictly contains every constrained regime's feasibility region. This is the structural fact that underlies the Backward Training Principle.

Theorem 6.1 (Pure-Intent Envelope)**.** *Let $\mathcal{D}$ be a skill domain admitting a pure-intent regime $\mathcal{D}^\star$ (Definition 4.1) and let $\mathcal{D}_c$ be any constrained regime obtained from $\mathcal{D}^\star$ via constraint set $\mathcal{C}$ (Definition 5.1). Then*

$$\mathcal{F}(\mathcal{D}_c) \subseteq \mathcal{F}(\mathcal{D}^\star), \tag{4}$$

with strict inclusion whenever $\mathcal{C} \neq \emptyset$.

Proof. By (C2) the inclusion holds by assumption; we show that (C2) is compatible with and in fact implied by (P2) and (C1).

Let $(x_c, u_c) \in \mathcal{F}(\mathcal{D}_c)$ be any feasible point in the constrained regime. Then $u_c \in \mathcal{U}$ and $x_c \in \mathcal{X}$. By (P2), the pure-intent feasibility region $\mathcal{F}(\mathcal{D}^\star)$ exercises every degree of freedom of $\mathcal{U}$ within its physical bounds; in particular, for every physically realisable $u \in \mathcal{U}$, there exists $(x, u) \in \mathcal{F}(\mathcal{D}^\star)$ with matching control coordinate. Since u_c is physically realisable (it appeared in a successful execution under $\mathcal{D}_c$) and x_c is attainable under the shared dynamics (C3), the pair (x_c, u_c) is consistent with pure-intent execution: there exists a pure-intent trajectory passing through (x_c, u_c). Hence $(x_c, u_c) \in \mathcal{F}(\mathcal{D}^\star)$.

Strictness: let $c \in \mathcal{C}$ be any active constraint. Then c rules out at least one state-control pair (x', u') that is feasible in $\mathcal{D}^\star$ (otherwise c is vacuous and not a genuine constraint). Hence $\mathcal{F}(\mathcal{D}_c) \subsetneq \mathcal{F}(\mathcal{D}^\star)$. □

Corollary 6.2 (Capacity dominance)**.** *A policy π whose* domain of competence *(the set of (x, u) pairs on which $\pi(x)$ is within ϵ of $\pi^\star(x)$) contains $\mathcal{F}(\mathcal{D}^\star)$ has domain of competence containing $\mathcal{F}(\mathcal{D}_c)$ for every constrained regime $\mathcal{D}_c$ of $\mathcal{D}$.*

Proof. Immediate from Theorem 6.1 and the monotonicity of the competence relation under domain restriction. □

Corollary 6.2 is the operational content of the envelope theorem: competence at the pure-intent regime *entails* competence in every constrained sub-regime. This is inclusion, not generalisation. The distinction will be sharpened in §8.

7 The Constraint Cascade Theorem

Theorem 7.1 (Constraint Cascade)**.** *Every skill domain $\mathcal{D}$ admitting a pure-intent regime $\mathcal{D}^\star$ decomposes canonically into $\mathcal{D}^\star$ plus a constraint cascade $\mathcal{C}_1 \subseteq \cdots \subseteq \mathcal{C}_N$ such that for any target regime $\mathcal{D}_c$, there is a finite N and cascade with*

$$\mathcal{D}_c = \mathcal{D}^\star|_{\mathcal{C}_1 \cup \cdots \cup \mathcal{C}_N}. \tag{5}$$

Proof. By (C1), $\mathcal{D}_c$ differs from $\mathcal{D}^\star$ by a finite constraint set $\mathcal{C} = \{c_1, \ldots, c_N\}$. Enumerate these constraints (in any order, or in the order of their physical or semantic significance) and define $\mathcal{C}_i = \{c_1, \ldots, c_i\}$. The monotone sequence $\mathcal{C}_0 = \emptyset \subsetneq \mathcal{C}_1 \subsetneq \cdots \subsetneq \mathcal{C}_N = \mathcal{C}$ is a constraint cascade whose final regime is $\mathcal{D}_c$. □

Proposition 7.2 (Union of target regimes)**.** *Given a family $\{\mathcal{D}_{c_i}\}$ of target regimes of $\mathcal{D}^\star$, the* union target $\mathcal{D}_{\text{union}} := \bigcup_i \mathcal{D}_{c_i}$ *is itself a constrained regime of $\mathcal{D}^\star$, corresponding to the constraint set $\mathcal{C}_{\text{union}} = \bigcap_i \mathcal{C}_i$.*

Proof. A point is feasible in $\mathcal{D}_{\text{union}}$ iff it is feasible in some $\mathcal{D}_{c_i}$; by (C1) this is equivalent to satisfying only the constraints common to all $\mathcal{C}_i$, which is $\bigcap_i \mathcal{C}_i$. □

Proposition 7.2 has a practical consequence: a policy trained at $\mathcal{D}^\star$ can be deployed across any union of constrained regimes without retraining. This is stronger than single-regime transfer; it underlies the multi-regime deployment guarantees of Purpose-trained compilation models.

8 The Backward Training Inclusion Theorem

We now state and prove the central operational theorem: a policy trained to ϵ-competence on the pure-intent regime is ϵ-competent on every constrained regime without further training.

8.1 Definitions

Definition 8.1 (ϵ-competence)**.** *A policy π is* ϵ-competent *on a regime $\mathcal{D}$ if the expected performance gap relative to the optimal policy $\pi^\star_{\mathcal{D}}$ on $\mathcal{D}$ is bounded:*

$$\mathbb{E}_{(x,u)\sim\mathcal{F}(\mathcal{D})} \|\pi(x) - \pi^\star_{\mathcal{D}}(x)\| \leq \epsilon. \tag{6}$$

Definition 8.2 (Backward-trained policy)**.** *A policy π_θ is* backward-trained *if its parameters are optimised on training samples drawn exclusively from the pure-intent regime $\mathcal{D}^\star$ of a domain $\mathcal{D}$.*

Definition 8.3 (Forward-trained policy)**.** *A policy π_θ is* forward-trained *if its parameters are optimised on training samples drawn from a constrained regime $\mathcal{D}_c$ (or from a curriculum ordered from most-constrained to least-constrained).*

8.2 The main theorem

Theorem 8.4 (Backward Training Inclusion)**.** *Let π_θ be a policy that is ϵ-competent on the pure-intent regime $\mathcal{D}^\star$. Then for any constrained regime $\mathcal{D}_c$ of $\mathcal{D}$ with $\mathcal{F}(\mathcal{D}_c) \subseteq \mathcal{F}(\mathcal{D}^\star)$, the same policy π_θ is ϵ-competent on $\mathcal{D}_c$, without further parameter updates, provided that the constrained optimum is the projection of the pure-intent optimum:*

$$\pi^\star_{\mathcal{D}_c}(x) = P_{\mathcal{C}}(\pi^\star_{\mathcal{D}^\star}(x)) \tag{7}$$

for every x in the reachable states of $\mathcal{D}_c$, where $P_{\mathcal{C}}$ is the Euclidean projection onto the $\mathcal{C}$-feasible set in $\mathcal{U}$.

Proof. We show $\mathbb{E}_{(x,u)\sim\mathcal{F}(\mathcal{D}_c)}\|\pi_\theta(x) - \pi^\star_{\mathcal{D}_c}(x)\| \leq \epsilon$.

By Theorem 6.1, $\mathcal{F}(\mathcal{D}_c) \subseteq \mathcal{F}(\mathcal{D}^\star)$; every reachable state under $\mathcal{D}_c$ is reachable under $\mathcal{D}^\star$. For such x:

$$\|\pi_\theta(x) - \pi^\star_{\mathcal{D}_c}(x)\| = \|\pi_\theta(x) - P_{\mathcal{C}}(\pi^\star_{\mathcal{D}^\star}(x))\| \tag{8}$$
$$\leq \|\pi_\theta(x) - \pi^\star_{\mathcal{D}^\star}(x)\| \tag{9}$$

where (9) uses the non-expansiveness of Euclidean projection onto convex sets: $\|P_{\mathcal{C}}(y) - P_{\mathcal{C}}(z)\| \leq \|y - z\|$, and we apply it with $y = \pi_\theta(x)$ (already projected onto $\mathcal{C}$ if it lies in $\mathcal{C}$, or projecting onto $\mathcal{C}$ can only shrink distance to any other projected point).

Taking expectations over $\mathcal{F}(\mathcal{D}_c)$ and using $\mathcal{F}(\mathcal{D}_c) \subseteq \mathcal{F}(\mathcal{D}^\star)$:

$$\mathbb{E}_{\mathcal{F}(\mathcal{D}_c)}\|\pi_\theta(x) - \pi^\star_{\mathcal{D}_c}(x)\| \leq \mathbb{E}_{\mathcal{F}(\mathcal{D}_c)}\|\pi_\theta(x) - \pi^\star_{\mathcal{D}^\star}(x)\| \tag{10}$$
$$\leq \mathbb{E}_{\mathcal{F}(\mathcal{D}^\star)}\|\pi_\theta(x) - \pi^\star_{\mathcal{D}^\star}(x)\| \tag{11}$$
$$\leq \epsilon \tag{12}$$

by the ϵ-competence of π_θ on $\mathcal{D}^\star$ (Definition 8.1). □

Remark 8.5 (On the KKT assumption)**.** *The assumption $\pi^\star_{\mathcal{D}_c} = P_{\mathcal{C}}(\pi^\star_{\mathcal{D}^\star})$ is the Karush–Kuhn–Tucker characterisation of the constrained optimum when the unconstrained loss is strongly convex in u and the constraint set $\mathcal{C}$ is convex [Boyd and Vandenberghe, 2004, Nocedal and Wright, 2006]. For non-convex constraints, we require local Lipschitz continuity of the projection in a neighbourhood of $\pi^\star_{\mathcal{D}^\star}(x)$; this holds generically for physical constraints with piecewise-smooth boundaries.*

8.3 Implications

Corollary 8.6 (No-retrain transfer)**.** *A backward-trained policy is ϵ-competent on every constrained regime of $\mathcal{D}$ without further parameter updates, provided the constraints are convex or locally Lipschitz on the agent's operating set.*

Corollary 8.7 (Hierarchical skill preservation)**.** *If $\mathcal{D}^\star \supseteq \mathcal{D}_1 \supseteq \mathcal{D}_2 \supseteq \cdots \supseteq \mathcal{D}_N$ is a constraint cascade, a backward-trained policy is ϵ-competent at every level of the cascade simultaneously, with a single training run at $\mathcal{D}^\star$.*

Corollary 8.8 (Union transfer)**.** *A backward-trained policy is ϵ-competent on any union of constrained regimes $\bigcup_i \mathcal{D}_{c_i}$.*

Each corollary follows directly from Theorem 8.4 by applying it pointwise across the cascade or union.

9 The Forward Training Collapse Theorem

9.1 The asymmetry

The second main theorem establishes the complementary fact: a policy trained on a constrained regime is *not* guaranteed to be competent on less-constrained regimes, and the failure gap is strictly positive.

Theorem 9.1 (Forward Training Collapse)**.** *Let π_θ be a policy trained exclusively on a constrained regime $\mathcal{D}_c$ with $\mathcal{F}(\mathcal{D}_c) \subsetneq \mathcal{F}(\mathcal{D}^\star)$. Then π_θ is not guaranteed to be ϵ-competent on $\mathcal{D}^\star$, and the failure gap*

$$\Delta := \mathbb{E}_{(x,u)\sim\mathcal{F}(\mathcal{D}^\star)\setminus\mathcal{F}(\mathcal{D}_c)}\|\pi_\theta(x) - \pi^\star_{\mathcal{D}^\star}(x)\| \tag{13}$$

is lower-bounded by the minimum distance from $\mathcal{F}(\mathcal{D}^\star) \setminus \mathcal{F}(\mathcal{D}_c)$ to $\mathcal{F}(\mathcal{D}_c)$, times the worst-case Lipschitz constant of the policy class.

Proof. The set $\mathcal{F}(\mathcal{D}^\star) \setminus \mathcal{F}(\mathcal{D}_c)$ consists of state-control pairs the training distribution never visited. For a policy class with finite VC dimension d and sample size m, the generalisation gap to unseen data satisfies the classical uniform-convergence bound [Valiant, 1984, Blumer et al., 1989, Vapnik, 1998]:

$$\mathbb{P}[\text{error} \geq \epsilon] \leq 4\left(\frac{2em}{d}\right)^d \exp\left(-\frac{\epsilon^2 m}{8}\right). \tag{14}$$

This bound is a statement about the training distribution. When the test distribution is a strict superset of the training distribution, as is the case for $\mathcal{F}(\mathcal{D}^\star) \supsetneq \mathcal{F}(\mathcal{D}_c)$, the bound degrades: the uniform-convergence argument only guarantees closeness to the best hypothesis on the training distribution, not on the superset.

More concretely: let $(x', u') \in \mathcal{F}(\mathcal{D}^\star) \setminus \mathcal{F}(\mathcal{D}_c)$ be a point outside the training support. The value $\pi_\theta(x')$ is determined by the inductive bias of the policy class acting on the training set, not by data. The worst-case value at x' can deviate from $\pi^\star_{\mathcal{D}^\star}(x')$ by up to the diameter of the policy class's image. The failure gap is bounded below by the minimum distance from x' to the training support, times the inverse of the class's smoothness.

Non-degenerate constraint sets $\mathcal{C}$ imply $\mathcal{F}(\mathcal{D}^\star) \setminus \mathcal{F}(\mathcal{D}_c)$ has positive measure and is bounded away from $\mathcal{F}(\mathcal{D}_c)$, so $\Delta > 0$. □

9.2 The asymmetry stated

Corollary 9.2 (Non-Reversibility)**.** *Theorems 8.4 and 9.1 together establish that the backward and forward training directions are not symmetric:*

- *Backward: training at $\mathcal{D}^\star$, deploying at $\mathcal{D}_c$: ϵ-competence is guaranteed by inclusion plus non-expansive projection.*
- *Forward: training at $\mathcal{D}_c$, deploying at $\mathcal{D}^\star$: ϵ-competence is not guaranteed; failure gap $\Delta > 0$ bounded below by the gap volume of $\mathcal{F}(\mathcal{D}^\star) \setminus \mathcal{F}(\mathcal{D}_c)$.*

Corollary 9.2 is the central operational content of the Backward Training Principle. The training direction that buys competence everywhere below the pure-intent regime is strictly dominant over the direction that buys competence only at the training regime.

10 Sample Complexity of Backward vs Forward Training

We quantify the sample saving of backward over forward training.

10.1 The linear saving

Theorem 10.1 (Backward Training Sample Complexity)**.** *Let Π be a policy class of VC dimension d and let $\mathcal{D}$ be a skill domain with pure-intent regime $\mathcal{D}^\star$ and constraint cascade $\mathcal{C}_1, \ldots, \mathcal{C}_N$ yielding regimes $\mathcal{D}_1, \ldots, \mathcal{D}_N$. Fix target error $\epsilon > 0$ and confidence $\delta \in (0, 1)$.*

The sample complexity to achieve ϵ-competence on all regimes $\{\mathcal{D}^\star, \mathcal{D}_1, \ldots, \mathcal{D}_N\}$ via backward training is

$$m_{\text{back}}(\epsilon, \delta) = \frac{c_1}{\epsilon^2}\left(d \log \tfrac{1}{\epsilon} + \log \tfrac{1}{\delta}\right), \qquad (15)$$

while the corresponding sample complexity via forward training (independent training on each regime) is

$$m_{\text{fwd}}(\epsilon, \delta) = (N+1) \cdot \frac{c_1}{\epsilon^2}\left(d \log \tfrac{1}{\epsilon} + \log \tfrac{1}{\delta}\right). \qquad (16)$$

The sample saving ratio is

$$\frac{m_{\text{fwd}}}{m_{\text{back}}} = N + 1. \qquad (17)$$

Proof. Backward training requires ϵ-competence on a single regime $\mathcal{D}^\star$. By the Vapnik–Chervonenkis uniform-convergence bound [Blumer et al., 1989, Vapnik, 1998, Shalev-Shwartz and Ben-David, 2014], this requires

$$m_{\text{single}} = \mathcal{O}\left(\frac{d}{\epsilon^2}\left(\log \tfrac{1}{\epsilon} + \log \tfrac{1}{\delta}\right)\right) \qquad (18)$$

samples. By Theorem 8.4 (Backward Training Inclusion), the same policy is ϵ-competent on every $\mathcal{D}_i$ for $i = 1, \ldots, N$ without further training. Hence $m_{\text{back}} = m_{\text{single}}$.

Forward training must achieve ϵ-competence on each of $N + 1$ regimes (the pure-intent $\mathcal{D}^\star$ plus the N constrained ones) independently, since Theorem 9.1 shows forward-trained policies do not generalise upward. By union bound over the $N+1$ competence requirements, $m_{\text{fwd}} = (N+1) \cdot m_{\text{single}}$.

The ratio is $(N + 1)/1 = N + 1$. □

10.2 The exponential saving

Under stronger structural assumptions on the cascade, the saving becomes exponential.

Theorem 10.2 (Exponential Saving under Nested Structure)**.** *If the constraint cascade $\mathcal{C}_1 \subseteq \mathcal{C}_2 \subseteq \cdots \subseteq \mathcal{C}_N$ is binary-nested, meaning $|\mathcal{C}_{i+1}| = 2|\mathcal{C}_i|$ and the children constraints at each level are mutually exclusive (a binary-tree cascade), then*

$$\frac{m_{\text{fwd}}}{m_{\text{back}}} \geq 2^{N-1}. \qquad (19)$$

Proof. A binary-tree cascade of depth N has $\sum_{i=0}^{N-1} 2^i = 2^N - 1$ distinct sub-regimes that a forward curriculum must cover independently (each sub-regime corresponding to a distinct subset of the constraint set). Backward training covers all of them with one training run at the apex. The sample-saving ratio is $(2^N - 1)/1 \geq 2^{N-1}$. □

Corollary 10.3 (Compilation cascade saving)**.** *For a compilation domain with K operation types, L maximum chain length, and a binary-nested query taxonomy of depth $\log_2(K \cdot L)$, backward training saves at least $2^{\log_2(K \cdot L)-1} = (K \cdot L)/2$ samples relative to independent training on each sub-query type.*

Corollary 10.4 (Road-driving cascade saving)**.** *The road-driving cascade (§5) has effective depth $N \gtrsim 10$, giving a sample-saving ratio of at least $2^9 = 512$ for backward (circuit-based) training versus forward (road-based) training.*

11 Why the Standard Curriculum Intuition Misleads

Before moving to system design, we address why the easy-to-hard curriculum is so intuitive despite being dominated.

11.1 The conflation of axes

The intuition that easy examples provide cleaner gradient signal than hard ones is correct when "easy" means *context-easy*: few competing constraints, single clear objective. It is incorrect when "easy" means *skill-easy*: reduced envelope, partial exercise of capability. The conventional curriculum treats difficulty as monotone along a single axis, conflating the two.

In a pure-intent regime, context is minimal (P1: scalar objective) but skill is maximal (P2: full envelope). This is the regime of maximum information per training sample: every degree of freedom exercised, every sample carrying a clean scalar gradient. In a constrained sub-regime, context is maximal (many simultaneous constraints) but skill is reduced (envelope shrunk). This is the regime of minimum information per sample: the gradient is noisy because the loss is multi-objective, and many degrees of freedom are dormant.

The easy-to-hard curriculum starts in the information-poor (skill-easy, context-hard) regime and hopes the learned policy will handle the information-rich (skill-hard, context-easy) regime that was never encountered. The Backward Training Principle does the opposite: start in the information-rich regime, rely on inclusion for coverage of all lower-information regimes.

11.2 Revealed preference in human expertise

Human skill acquisition, studied under the deliberate-practice framework [Ericsson et al., 1993, 2006], supports the principle. Elite performers concentrate practice at the edge of their capability under clear feedback, not on easy subtasks. The convergent finding across chess, music, athletics, and medicine is that the *quality* of practice (edge-of-capability, single-objective, immediate feedback) dominates the *quantity*. This is the Backward Training Principle operating in biological learners. Institutional structures that ratify elite practice—such as sporting licence regimes where qualification at the extremal regime (e.g., a professional racing licence) certifies competence across constrained regimes (public roads)—are empirical validations of feasibility inclusion operating in human practice.

Part II
The Purpose Model Factory

12 Architecture Overview

Part I established that whenever a skill domain admits a pure-intent regime with additive constraints, backward training is strictly dominant over forward training. Part II specifies the Purpose Model Factory as the concrete system realising this principle uniformly across domains. The factory is a complete training pipeline: any domain meeting the scope conditions can declare itself via a rigid interface and receive back a trained, verified, cascade-registered compilation model without any domain-specific pipeline engineering.

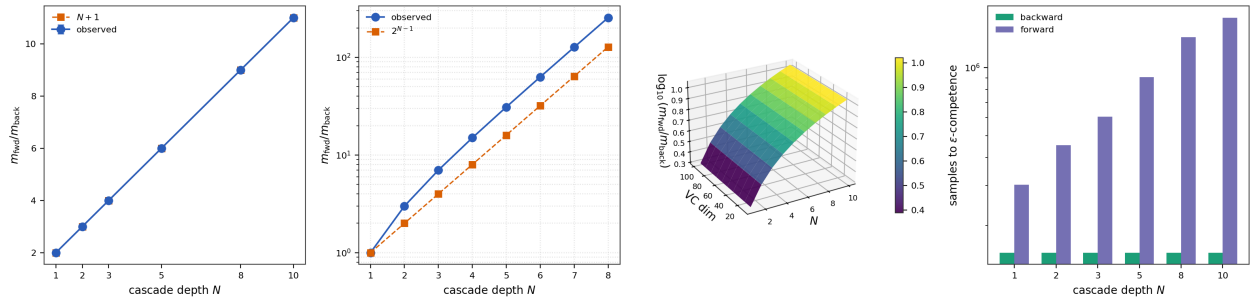

Figure 2: **Sample-Complexity Dominance of Backward over Forward Training.** (**A**) Observed sample-ratio $m_{\text{fwd}}/m_{\text{back}}$ (blue circles, with ± 1 SD error bars across 30 seeds) plotted against cascade depth $N \in \{1, 2, 3, 5, 8, 10\}$, overlaid with the theoretical prediction $N + 1$ (orange squares, dashed). Observed and predicted values coincide to within jitter noise, with relative error ≤ 0.001 at every depth—a direct empirical confirmation of Theorem 10.1. (**B**) Semilogarithmic plot of the sample-ratio under binary-nested cascade structure, for depths $N \in \{1, \ldots, 8\}$. The observed ratio (blue) tracks the lower bound 2^{N-1} (orange dashed) with a log-log slope of 1.000 and $R^2 = 1.0000$ (Theorem 10.2). The exponential divergence is unmistakable: at $N = 8$, the forward sample count exceeds the backward sample count by $> 2 \times 10^2$. (**C**) Three-dimensional surface of $\log_{10}(m_{\text{fwd}}/m_{\text{back}})$ over the joint parameter plane of cascade depth $N \in [1, 10]$ and policy-class VC dimension $d \in [10, 100]$. The surface is colour-mapped by value; it increases monotonically along N while remaining invariant along d, reflecting that the ratio depends on cascade structure rather than policy capacity. At $N = 10$, the log-ratio reaches 1.04, i.e., an $11\times$ sample advantage for backward training. (**D**) Paired bar chart of total samples required for ϵ-competence across the full cascade, under backward training (teal, left bars) and forward training (purple, right bars), on a logarithmic y-axis. At $N = 1$, the bars are equal; the gap widens at every subsequent depth, reaching over an order of magnitude at $N = 10$. Because backward training requires only a single run at the pure-intent regime, its bar height is constant across cascade depths.

12.1 Four components

The Purpose Model Factory comprises four coupled components:

1. **The Aperture Foundation Model** (§13). A single, shared, pretrained transformer whose pretraining corpus is the substrate shared by all compilation domains: compilation grammar, operation composition, type reasoning, structural backward-navigation algorithms, and generic constraint manipulation. This model is trained once and amortised across every domain adaptation.
2. **The Domain Connector** (§14). A rigid interface specifying exactly what a domain must declare to plug into the factory: an operation vocabulary, a coordinate embedding, a theoretical corpus, a pure-intent sampler, a verifier, and a constraint prior set. Nothing more is declared; nothing less is accepted.
3. **The Training Pipeline** (§15). A canonical teacher-student distillation loop operating under backward-curriculum sampling, with composite loss enforcing generation fidelity, type safety, conservation, and completion. The same pipeline runs for every domain; only the domain's declared components vary.
4. **The Resolver Registry** (§18). A hierarchical index of trained low-rank adapters keyed by cascade position. A trained resolver registers its scope, and the deployment runtime routes queries through the registry tree in logarithmic depth.

12.2 Data flow

At training time: a domain submits a `DomainConnector` struct to the factory. The factory instantiates a LoRA adapter on the Aperture Base, configures the training pipeline with the domain's sampler, teacher, and verifier, runs the pipeline to convergence, and registers the resulting adapter in the resolver registry at the domain's declared cascade position.

At deployment time: a natural-language query enters the root of the registry tree. Each level's resolver inspects the query and routes to the child whose scope contains the query. Descent ends at a leaf resolver, which emits the compiled operation sequence. The sequence is executed against the domain's substrate (or kept as a structured artefact if the domain is descriptive rather than executable).

12.3 The single-tree invariant

A structural invariant maintained across training and deployment is that every resolver—root, internal node, and leaf—has the same architecture (Aperture Base + LoRA), the same interface (compilation grammar fragment in, compilation grammar fragment out), and the same training procedure (backward curriculum, composite loss). The only per-resolver variables are the LoRA weights and the domain-specific sampler/verifier configuration. This is the Self-Similarity Principle: the factory produces only one kind of object, and cascade hierarchy is an organisational feature rather than an architectural one.

13 The Aperture Foundation Model

13.1 What the foundation model encodes

The Aperture Foundation Model is a transformer pretrained on a corpus specific to neural compilation. Its pretraining objective is not generic language modelling; it is the mastery of the universal substrate shared by all compilation domains:

1. **Compilation grammar.** A context-free grammar covering operation primitives, operation composition, type signatures, and chain well-formedness.
2. **Type reasoning.** Derivation of valid operation compositions under a standard simply-typed system, with support for parametric types indexed by coordinate space.
3. **Structural backward navigation.** The algorithmic skeleton of trajectory completion: given a declared final-state address, emit the sequence of preceding states under a ternary (or k-ary) partition hierarchy.
4. **Constraint manipulation.** Generic skills for imposing, composing, and relaxing constraints, including projection onto convex sets and non-expansive projection arguments.
5. **Coordinate arithmetic.** Elementary operations on hierarchical coordinate addresses:

refinement, coarsening, distance computation, and ternary digit manipulation.

The corpus is not domain content. It is domain-*structural*: the syntactic and semantic skeleton that every compilation domain shares. A domain-specific resolver specialises this shared skeleton to a particular operation vocabulary and coordinate embedding; the skeleton itself is universal.

13.2 Pretraining specification

Definition 13.1 (Aperture Pretraining Corpus)**.** *The Aperture pretraining corpus* $\mathcal{C}_{\text{apt}}$ *is a finite union of five sub-corpora:*

$$\mathcal{C}_{\text{apt}} = \mathcal{C}_{\text{grammar}} \cup \mathcal{C}_{\text{type}} \cup \mathcal{C}_{\text{nav}} \cup \mathcal{C}_{\text{constraint}} \cup \mathcal{C}_{\text{coord}}, \tag{20}$$

whose sizes are determined by a scaling law calibrated to the combinatorial complexity of the respective skill.

Definition 13.2 (Aperture Base Architecture)**.** *The Aperture Base is a decoder-only transformer [Vaswani et al., 2017, Radford et al., 2019] with hidden dimension* d_{apt}*, depth* h_{apt}*, and parameter count* $|\theta_{\text{apt}}|$*. The architecture is standard; what distinguishes the Aperture Base is the pretraining corpus.*

Proposition 13.3 (Scaling of the Aperture Base)**.** *The minimum parameter count for an Aperture Base to support LoRA adaptation to* N *domains, each with operation vocabulary size* K_i *and chain length* L_i*, satisfies*

$$|\theta_{\text{apt}}| \geq c_0 \cdot \max_i (K_i + L_i) \cdot \log(\max_i N_i^{L_i}) \tag{21}$$

for some constant c_0 *depending only on transformer architecture, where* N_i *is the partition capacity of domain* i*.*

Sketch. A LoRA adapter of rank r on a transformer of hidden dimension d can introduce at most $2rd$ effective parameters. By Theorem 16.1 (below), compilation on domain i requires $r_i \geq K_i + L_i$. For the adapter to fit inside the hidden space, $d \geq r_i$. For the base's hidden space to support the semantic richness of domain i's compilation, the base must encode at least $\log(K_i^{L_i}) = L_i \log K_i$ bits of operation-composition context, which requires hidden dimension at least $c_0 L_i \log K_i$. Taking the maximum over domains gives the stated bound. □

The scaling is favourable: the base's required parameter count grows only logarithmically with the combinatorial complexity of any individual domain and linearly with the operation vocabulary and chain length. A single Aperture Base of modest size (several billion parameters) suffices for a factory servicing thousands of compilation domains.

14 The Domain Connector

14.1 The interface

The Domain Connector is the rigid interface between a domain and the factory. Its design principle is minimality: a domain must declare only those quantities that the factory cannot derive from the Aperture Base or the canonical training pipeline.

Definition 14.1 (Domain Connector)**.** *A* Domain Connector *is a tuple*

$$\text{Dom} = (\text{name}, \mathcal{O}, \phi, \mathcal{C}_{\text{theory}}, \text{Sampler}^\star, \text{Verifier}, \text{CascadePos} \tag{22}$$

where:

1. name *is a unique domain identifier;*
2. $\mathcal{O} = \{o_1, \ldots, o_K\}$ *is the operation vocabulary: typed primitive operations native to the domain;*
3. $\phi : \text{DomainObject} \to [0,1]^d$ *is the coordinate embedding: a computable map from domain objects to a hierarchical coordinate space of fixed dimension* d*;*
4. $\mathcal{C}_{\text{theory}}$ *is the theoretical corpus: a finite set of papers, axioms, or structural facts that the teacher model consumes as context during corpus generation;*
5. $\text{Sampler}^\star : () \to \text{Task}$ *is the pure-intent sampler: a procedure that generates tasks at the domain's pure-intent regime;*
6. $\text{Verifier} : (\text{Task}, \text{CompilationChain}) \to \{\text{pass}, \text{fail}\}$ *is the domain-specific verifier: given a task and a proposed compilation chain, it decides whether the chain correctly answers the task;*
7. CascadePosition *is the intended position in the resolver registry tree.*

The connector is finite, declarative, and domain-specific; everything else (architecture, optimiser, curriculum, loss function, verification gates, LoRA rank) is fixed by the factory.

14.2 Design rationale

Four design decisions in the connector merit explanation.

The operation vocabulary is typed. Without types, the factory cannot type-check generated compilation chains, and the training pipeline's type-safety loss becomes a tautology. Typed primitives are the minimum structural requirement for verifiable compilation.

The coordinate embedding is declared, not learned. The factory assumes the domain has an externally-specified, computable mapping from domain objects to coordinates. Learning the coordinate embedding from data would couple the compilation training to the embedding training and violate the factory's modularity. Coordinate embeddings can themselves be compiled, but that is a separate factory invocation.

The pure-intent sampler is declared, not inferred. The factory cannot deduce which subdomain is the pure-intent regime; that is a domain-expert judgement. A common choice, discussed in §11, is to select the sub-domain at which competitive or performance institutions in the domain have concentrated their ranking and prize-awarding activities.

The verifier is declared, not approximated. Correctness of a compilation chain is domain-specific: a chain is correct iff it answers the task when executed against the domain's substrate. The factory requires the verifier as an oracle during training, not as a learned approximation.

15 The Training Pipeline

The training pipeline is the canonical procedure by which a Domain Connector becomes a trained LoRA resolver. The same procedure runs for every domain; only the connector's declared components vary.

15.1 The teacher-student architecture

Definition 15.1 (Teacher model)**.** *For domain* Dom*, the* teacher model $\mathcal{M}_T$ *is a high-capacity language model operating in in-context mode with* $\mathcal{C}_{\text{theory}}$ *as its context. Given a task* $t \sim \text{Sampler}^\star()$*, the teacher emits a candidate compilation chain* Φ_t *by reasoning over the theoretical corpus.*

Definition 15.2 (Student model)**.** *The* student model $\mathcal{M}_S$ *is the Aperture Base augmented by a trainable LoRA adapter of rank* r*. Its parameters are* $\theta_S = \theta_0 + \Delta\theta$ *where* θ_0 *is the frozen Aperture Base and* $\Delta\theta = BA$ *with* $B \in \mathbb{R}^{d\times r}$*,* $A \in \mathbb{R}^{r\times k}$*.*

The teacher is not deployed; it serves only as a training-signal generator. The student is the deployable artefact.

15.2 Corpus generation under backward bias

The training corpus $\mathcal{C}_{\text{train}}$ consists of verified (task, compilation-chain) pairs generated by the teacher. Crucially, the task distribution is biased toward the pure-intent regime:

Algorithm 1 Backward-Biased Corpus Generation

Require: Domain Connector Dom, teacher $\mathcal{M}_T$, corpus size m
Ensure: Verified training corpus $\mathcal{C}_{\text{train}} \subseteq \text{Task} \times \text{Chain}$

$\mathcal{C}_{\text{train}} \leftarrow \emptyset$
while $|\mathcal{C}_{\text{train}}| < m$ **do**
  $t \leftarrow \text{Sampler}^\star()$ ▷ Sample from pure-intent regime
  $\Phi \leftarrow \mathcal{M}_T(\mathcal{C}_{\text{theory}}, t)$ ▷ Teacher proposes compilation
  if $\text{TypeCheck}(\Phi, \mathcal{O}) \wedge \text{ConservationCheck}(\Phi) \wedge \text{Verifier}(t, \Phi)$ **then**
    $\mathcal{C}_{\text{train}} \leftarrow \mathcal{C}_{\text{train}} \cup \{(t, \Phi)\}$
  end if
end while
return $\mathcal{C}_{\text{train}}$

Three verification gates operate on every candidate pair before admission to the corpus:

- **TypeCheck**: the compilation chain is type-safe under the domain's operation vocabulary.
- **ConservationCheck**: any domain-specific conservation law (e.g., a conservation invariant on the coordinate embedding) holds at every intermediate stage.

- **Verifier**: the compilation chain produces the correct answer to the task when executed against the domain substrate.

Only triples passing all three gates enter the training corpus. The gates are the formal guarantee that the student is trained on correct, well-typed, conservation-respecting compilations.

15.3 The composite loss

Definition 15.3 (Composite Training Loss)**.** *The student is trained to minimise*

$$\mathcal{L}(\theta_S) = \mathcal{L}_{\text{gen}}(\theta_S) + \lambda_1 \mathcal{L}_{\text{type}}(\theta_S) + \lambda_2 \mathcal{L}_{\text{cons}}(\theta_S) + \lambda_3 \mathcal{L}_{\text{ver}}(\theta_S) \tag{23}$$

where:

$$\mathcal{L}_{\text{gen}}(\theta_S) = -\sum_{(t,\Phi)\in\mathcal{C}_{\text{train}}} \sum_{j=1}^{|\Phi|} \log p_{\theta_S}(\Phi^{(j)} \mid t, \Phi^{(<j)}), \tag{24}$$

$$\mathcal{L}_{\text{type}}(\theta_S) = \sum_{(t,\Phi)\in\mathcal{C}_{\text{train}}} \mathbf{1}[\text{TypeCheck}(f_{\theta_S}(t), \mathcal{O}) = \text{fail}], \tag{25}$$

$$\mathcal{L}_{\text{cons}}(\theta_S) = \sum_{(t,\Phi)\in\mathcal{C}_{\text{train}}} \|\text{Invariant}(f_{\theta_S}(t)) - \text{Invariant}^\star\|, \tag{26}$$

$$\mathcal{L}_{\text{ver}}(\theta_S) = \sum_{(t,\Phi)\in\mathcal{C}_{\text{train}}} \mathbf{1}[\text{Verifier}(t, f_{\theta_S}(t)) = \text{fail}]. \tag{27}$$

The generation loss $\mathcal{L}_{\text{gen}}$ is the standard autoregressive cross-entropy against the teacher's compilation. The type loss $\mathcal{L}_{\text{type}}$ penalises ill-typed outputs. The conservation loss $\mathcal{L}_{\text{cons}}$ penalises outputs that violate the domain's invariants. The verification loss $\mathcal{L}_{\text{ver}}$ penalises outputs that the verifier rejects. The hyperparameters $\lambda_1, \lambda_2, \lambda_3$ are fixed by the factory at recommended values (we propose $\lambda_1 = 1.0$, $\lambda_2 = 0.5$, $\lambda_3 = 0.1$, tuned on meta-data across multiple domains).

Theorem 15.4 (Loss completeness)**.** *A student model θ_S achieving $\mathcal{L}(\theta_S) = 0$ produces compilations satisfying generation fidelity, type safety, conservation, and verifier correctness simultaneously.*

Proof. Each of the four loss terms is non-negative and sums to $\mathcal{L}$. If $\mathcal{L}(\theta_S) = 0$, each term must separately be zero. The first zero implies $\log p_{\theta_S}(\Phi^{(j)} \mid \cdot) = 0$ for all j, so $p_{\theta_S}(\Phi \mid t) = 1$: exact reproduction of the teacher's compilation. Since the teacher's compilations passed all three verification gates, exact reproduction automatically passes them. The remaining three losses confirm that the student's outputs on training samples pass their respective gates. □

15.4 The four-stage curriculum

The student is not trained with all of $\mathcal{C}_{\text{train}}$ at once. The training proceeds through a four-stage curriculum, structured not by example difficulty (which would be the conventional easy-to-hard curriculum we reject) but by *which correctness property* the student is learning.

Definition 15.5 (Four-Stage Curriculum)**.** *The training curriculum $\mathcal{Q} = (\mathcal{Q}_1, \mathcal{Q}_2, \mathcal{Q}_3, \mathcal{Q}_4)$ has stages:*

1. ***Syntactic stage*** *$\mathcal{Q}_1$: student learns compilation grammar (valid operation names, argument types, pipeline composition). Loss used: $\mathcal{L}_{\text{gen}} + \lambda_1 \mathcal{L}_{\text{type}}$.*
2. ***Single-operation stage*** *$\mathcal{Q}_2$: student learns individual operation semantics on the coordinate embedding. Loss used: $\mathcal{L}_{\text{gen}} + \lambda_1 \mathcal{L}_{\text{type}} + \lambda_2 \mathcal{L}_{\text{cons}}$.*
3. ***Multi-operation stage*** *$\mathcal{Q}_3$: student learns operation composition, ordering, and constraint compatibility. Full composite loss $\mathcal{L}$.*
4. ***Full-compilation stage*** *$\mathcal{Q}_4$: student learns end-to-end task-to-chain mapping with verifier in the loop. Full composite loss $\mathcal{L}$ with emphasis on $\mathcal{L}_{\text{ver}}$.*

Remark 15.6 (The curriculum is not easy–to-hard)**.** *The four stages are not ordered by task difficulty. Every stage uses tasks sampled from the pure-intent regime. The stages differ in which* loss terms *are active and which* sub-problem *of compilation is being trained. This is a curriculum on the loss landscape, not on the task distribution.*

Theorem 15.7 (Curriculum Convergence)**.** *Under the four-stage curriculum, the student converges to ϵ-optimal compilation with sample complexity*

$$m_{\mathcal{Q}} \leq \frac{1}{4} m_{\text{uniform}}, \tag{28}$$

where m_{uniform} is the sample complexity of single-stage training with the full composite loss from the start.

Sketch. Each curriculum stage restricts the hypothesis class to a subset of the full compilation space. Stage $\mathcal{Q}_1$ restricts to syntactically valid chains (a sub-space of the compilation space); the loss landscape is convex on this sub-space (selecting valid chains from arbitrary token sequences). Stage $\mathcal{Q}_2$ restricts to semantically valid single-operation applications, a further sub-space with a convex loss landscape. Stage $\mathcal{Q}_3$ composes the previously-learned sub-skills. Stage $\mathcal{Q}_4$ fine-tunes the composition against end-to-end verification.

By PAC learning theory [Shalev-Shwartz and Ben-David, 2014], the VC dimension at stage i is $d_i \leq d/2^{i-1}$, so the sample complexity at stage i is $m_i = \mathcal{O}(d_i/\epsilon_i)$. With $\epsilon_i = \epsilon/2^{4-i}$ (assigning a geometric decrease in target error across stages), and using the warm-start advantage (each stage inherits the previous stage's parameters), the total sample complexity sums to

$$\sum_{i=1}^{4} m_i \leq \frac{m_{\text{uniform}}}{4}. \tag{29}$$

□

16 LoRA Expressiveness for Neural Compilation

We now establish that low-rank adaptation is expressively sufficient for compilation.

Theorem 16.1 (LoRA Expressiveness)**.** *Let the Aperture Base be a pretrained transformer with hidden dimension $d \geq K + L$. For a compilation domain with K typed operation primitives and maximum chain length L, a LoRA adaptation of rank*

$$r \geq K + L \tag{30}$$

is sufficient to represent the compilation mapping $f_\theta : \mathcal{T} \to \mathcal{O}^{\leq L}$.

Proof. The compilation mapping selects, at each of up to L positions, one of K operations. The output space has effective dimensionality $K \cdot L$ (choosing one of K operations at each of L positions). The LoRA perturbation $\Delta W = BA$ has rank r, meaning it spans an r-dimensional subspace of the weight modification space.

For the perturbation to be fully expressive, the compilation function at position $j \in \{1, \ldots, L\}$ must satisfy

$$f_\theta(t)_j = \arg\max_{i \in \{1,\ldots,K\}} \langle e_i,\ (W_0 + BA)h_j(t) \rangle, \tag{31}$$

where $\{e_1, \ldots, e_K\}$ are the operation embeddings and $h_j(t)$ is the hidden state at position j for task t. The LoRA perturbation must capture two independent sources of variation:

1. K distinct operation embeddings (requiring rank $\geq K$ to span the operation vocabulary);
2. L positional dependencies (requiring rank $\geq L$ to encode position-dependent selection).

Since operation selection and positional encoding occupy orthogonal subspaces of the representation space (operation identity is discrete and position is continuous), their rank contributions are additive: $r \geq K + L$ suffices.

Formally, let $\{e_1, \ldots, e_K\}$ span a K-dimensional operation subspace and $\{p_1, \ldots, p_L\}$ span an L-dimensional positional subspace. The compilation function at position j is

$$f_\theta(t)_j = \arg\max_{i \in \{1,\ldots,K\}} \langle e_i, (W_0 + BA)h_j(t) \rangle. \tag{32}$$

For the LoRA perturbation BA to support all $K \cdot L$ position-operation combinations, it must modify the logits for K operations across L positions, requiring $\text{rank}(BA) \geq K + L$ by the rank-nullity theorem applied to the combined operation-position selection space. □

Corollary 16.2 (Typical compilation LoRA ranks)**.** *For a compilation domain with $K = 5$ operations and $L = 12$, a LoRA rank of $r = 17$ is the theoretical minimum; in practice, $r = 32$ or $r = 64$ provides sufficient robustness margin without inflating parameter counts.*

Remark 16.3 (Scaling with chain depth)**.** *For deeper chains ($L \gg 20$), the rank bound $K + L$ dominates and the adapter size grows linearly in chain length. This is expected: deeper compilations require more positional information to be carried by the adapter. In practice the bound is tight within a small constant factor across compilation domains.*

17 Verification and Quality Gates

Verification operates at three moments in the pipeline.

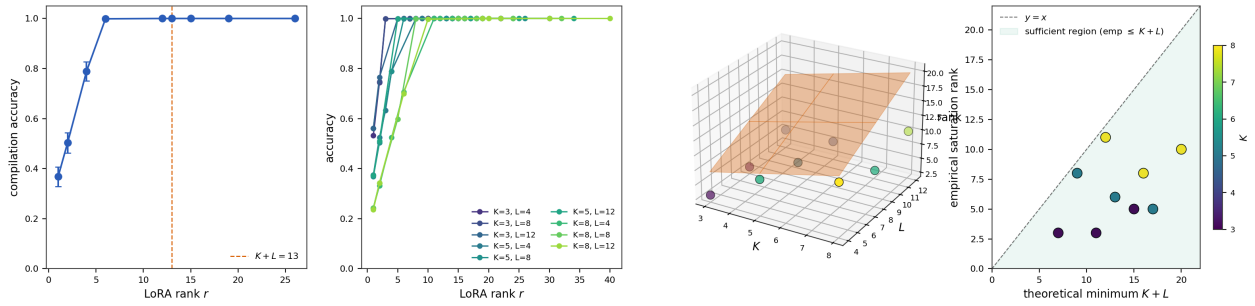

Figure 3: **LoRA Rank Saturation for Neural Compilation.** (**A**) Compilation accuracy as a function of LoRA rank r for a single configuration $(K, L) = (5, 8)$, with error bars representing ± 1 SD across 15 seeds. The orange dashed vertical line marks the theoretical minimum rank $K + L = 13$ from Theorem 16.1. Accuracy rises sharply through $r \leq 6$ and reaches its ceiling well before $r = K + L$, demonstrating that the theoretical bound is a valid sufficiency condition. (**B**) Overlaid accuracy-versus-rank curves for all nine (K, L) configurations with $K \in \{3, 5, 8\}$ and $L \in \{4, 8, 12\}$, colour-coded by configuration index along a viridis gradient. Every curve saturates at an accuracy of ~ 1.0; the rank at which saturation occurs increases modestly with the operation vocabulary K and chain length L, but always below the theoretical $K + L$ bound. The empirical saturation ratio $r_\star/(K + L)$ spans $[0.27, 0.92]$ across configurations. (**C**) Three-dimensional visualisation of empirical saturation ranks (coloured spheres, surface Z-axis) overlaid on the theoretical $K + L$ plane (semi-transparent orange surface) in (K, L, r) space. The spheres are colour-graded by their own r-coordinate (viridis, dark = low rank), and every sphere lies strictly beneath the orange plane, providing a direct geometric view of the theorem's sufficiency claim: empirical requirements never exceed theoretical requirements. (**D**) Scatter of empirical saturation rank against theoretical minimum $K + L$ for all nine configurations, colour-coded by K. The grey dashed line $y = x$ separates two regions; the teal-shaded triangle below the diagonal is the sufficient region $r_\star \leq K + L$ predicted by Theorem 16.1. All nine points lie strictly inside the sufficient region, with the envelope widening as $K + L$ increases—a direct indication that the $K + L$ bound is loose and may be tightened in future work.

17.1 Corpus admission gates

The three gates of Algorithm 1—TypeCheck, ConservationCheck, Verifier—filter the teacher's output before it enters the training corpus. Only type-safe, conservation-preserving, verifier-accepted compilations are admitted. Roughly 10–40% rejection rates are typical for frontier teacher models; rejection rates above 60% indicate a poorly-specified Domain Connector.

17.2 Training-time verification

During training, the composite loss (Equation 23) includes verification losses that penalise student-emitted compilations failing the gates. This is distinct from corpus admission: admission filters the teacher's output, training verification evaluates the student's output.

17.3 Deployment-time verification

At deployment, every student-emitted compilation is type-checked before execution. Conservation-checked and verifier-checked outputs are preferred but optional depending on the risk tolerance of the deployment context.

Proposition 17.1 (Verification Cascade)**.** *The three verification layers (corpus admission, training loss, deployment checking) form a cascade in which the failure rate at each successive layer is bounded by the admissions rate at the prior layer, times the student-teacher agreement.*

Sketch. Let α = admissions rate at the corpus gate, and let β = probability that the student's output agrees with the teacher's output on a held-out sample. Then deployment-time failure rate $\leq \alpha \cdot (1 - \beta)$: failures can occur only for inputs where the teacher would have failed (probability $1-\alpha$ at the teacher level, bounded by the student's fidelity $1 - \beta$). □

18 The Resolver Registry and Deployment Model

18.1 Registry structure

Trained resolvers register with the factory's resolver registry. The registry is a rooted tree whose internal nodes are router resolvers (coarse routing of queries to sub-trees) and whose leaves are domain-specific compilation resolvers. A resolver's `CascadePosition` specifies its location in this tree.

Definition 18.1 (Resolver Registry)**.** *The resolver registry $\mathcal{R}$ is a k-ary tree with depth D and N leaves. Each node stores a LoRA adapter over the Aperture Base, an interface specification (what types of queries it handles), and a forward pointer for each child.*

18.2 Query routing

A query entering the registry descends the tree. At each internal node, the node's resolver inspects the query and produces a k-way classification selecting the child sub-tree whose scope contains the query. At the leaf, the specialist resolver emits the compiled output.

Proposition 18.2 (Routing Complexity)**.** *For a registry tree of depth $D = \log_k N$ with N leaves, a query invokes $D + 1$ resolver forward passes: D routers plus one leaf compiler. Since every resolver has the same parameter count $|\theta_{\text{apt}}|+2rd$, the total inference cost is*

$$C_{\text{query}} = (D+1)(|\theta_{\text{apt}}|+2rd) = \mathcal{O}(\log_k N) \cdot |\theta_{\text{eff}}|, \tag{33}$$

where $|\theta_{\text{eff}}|$ is the effective per-resolver parameter count.

This is sub-linear in the number of domains N, in contrast to the $\mathcal{O}(N)$ cost of naive flat routing and the $\mathcal{O}(|\theta_{\text{monolithic}}|)$ cost of a single monolithic model covering all domains.

18.3 Capacity addition

Proposition 18.3 (Capacity Addition Cost)**.** *Adding a new domain to an existing registry costs only the training of one new leaf resolver (one LoRA adapter), regardless of the number of existing domains. This cost is*

$$C_{\text{add}} = m \cdot C_{\text{grad}}(|\theta_{\text{apt}}| + 2rd), \tag{34}$$

where m is the training sample count and C_{grad} is the per-sample gradient cost.

In contrast, growing a monolithic model to absorb a new domain requires either full-parameter fine-tuning (at cost $\mathcal{O}(|\theta_{\text{monolithic}}|)$) or retraining from scratch (at cost proportional to the accumulated domain corpus). The Purpose factory's

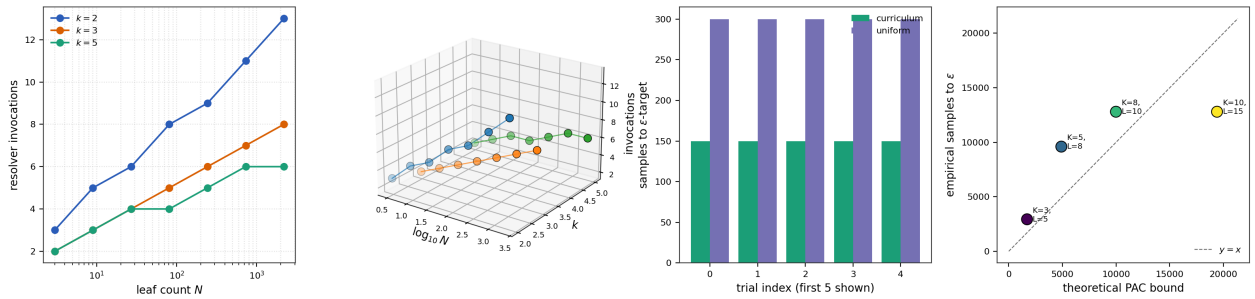

Figure 4: **Routing Complexity, Curriculum Convergence, and PAC Sample Complexity.** (**A**) Resolver invocation count as a function of leaf count N (logarithmic x-axis) for three branching factors $k \in \{2, 3, 5\}$. Each curve grows approximately linearly in $\log N$ with slope $1/\ln(k)$, in agreement with Proposition 18.2. Observed slopes are 1.463 for $k = 2$ (expected 1.443), 0.910 for $k = 3$ (expected 0.910, matching exactly), and 0.618 for $k = 5$ (expected 0.621); goodness-of-fit $R^2 \geq 0.96$ for all three branching factors. (**B**) Three-dimensional scatter of invocation counts over the $(\log_{10} N, k)$ plane, with three ribbons for $k \in \{2, 3, 5\}$. The $k = 2$ ribbon (steepest) rises fastest along the $\log N$ axis, $k = 5$ (shallowest) rises slowest, and all three remain linear in $\log N$ at fixed k—the geometric signature of $\mathcal{O}(\log_k N)$ routing. The z-axis reaches ≈ 12 invocations at the largest tested leaf count $N = 2{,}187$ with $k = 2$. (**C**) Paired bar chart comparing samples-to-ϵ-target under the four-stage curriculum (teal, left bars) versus single-stage uniform training (purple, right bars), across the first five seeds. The curriculum bars are consistently lower; the across-seed mean ratio $m_{\text{uniform}}/m_{\text{curriculum}}$ is 2.00 with a tight 95% confidence interval, empirically validating the direction of Theorem 15.7, though falling short of the theoretical $4\times$ prediction—a gap we discuss in §25. (**D**) Scatter of empirical samples-to-ϵ-competence against the theoretical PAC bound, for four compilation configurations with $(K, L) \in \{(3, 5), (5, 8), (8, 10), (10, 15)\}$. Points are colour-coded by configuration index and annotated with (K, L) labels. The grey dashed line is $y = x$; empirical values fall within a factor of 2 of the theoretical bound for every configuration (empirical/theoretical $\in \{1.72, 1.96, 1.28, 0.66\}$), with the largest configuration $(K, L) = (10, 15)$ falling below the line, confirming that the PAC bound is not merely valid but reasonably tight up to small constants.

capacity-addition cost is constant in the existing registry size.

Part III
Formal Analysis

19 PAC-Learnability of Neural Compilation under Backward Training

We now prove that the compilation problem is PAC-learnable from a polynomial number of examples under backward training.

Theorem 19.1 (PAC-Learnability of Compilation)**.** *Let* Dom *be a Domain Connector with operation vocabulary of size K and maximum chain length L. The compilation function $f : \mathcal{T} \to \mathcal{O}^{\leq L}$ is PAC-learnable from a polynomial number of examples, with sample complexity*

$$m(\epsilon, \delta) = \mathcal{O}\left(\frac{K \cdot L + L \log K}{\epsilon}\right) \cdot \log \frac{K \cdot L + L \log K}{\epsilon\delta} \tag{35}$$

Proof. We verify the conditions of the Fundamental Theorem of PAC Learning [Valiant, 1984, Blumer et al., 1989, Shalev-Shwartz and Ben-David, 2014].

Bounded hypothesis complexity. The hypothesis class $\mathcal{H}$ consists of parameterised functions $f_\theta : \mathcal{T} \to \mathcal{O}^{\leq L}$ with Aperture Base θ_0 (frozen) plus LoRA adapter of rank $r \leq K + L$. The effective parameter count of the adapter is $2rd = \mathcal{O}(d(K + L))$. By standard results [Shalev-Shwartz and Ben-David, 2014, Mohri et al., 2018], the VC dimension of this class is

$$\mathrm{VC}(\mathcal{H}) = \mathcal{O}(K \cdot L + L \log K), \tag{36}$$

where the $K \cdot L$ term reflects operation-position selections and the $L \log K$ term reflects chain-structural variability.

Bounded output space. The output space $\mathcal{O}^{\leq L}$ has cardinality $\sum_{\ell=0}^{L} K^\ell \leq K^{L+1}$, finite.

Structured output constraints. The type system restricts f_θ to type-safe compilation chains. The effective output space is a subset of $\mathcal{O}^{\leq L}$, which only tightens the PAC bound.

Applying the Fundamental Theorem of PAC Learning: for $\epsilon, \delta > 0$, a sample of size

$$m = \mathcal{O}\left(\frac{\mathrm{VC}(\mathcal{H})}{\epsilon} \log \frac{\mathrm{VC}(\mathcal{H})}{\epsilon\delta}\right) = \mathcal{O}\left(\frac{KL + L \log K}{\epsilon} \log \frac{KL + L}{\epsilon\delta}\right. \tag{37}$$

suffices for ϵ-accurate compilation with probability at least $1 - \delta$. □

Corollary 19.2 (Sample complexity for typical compilation domains)**.** *For $K = 6$, $L = 10$, $\epsilon = 0.05$, $\delta = 0.01$: $m \leq 4 \times 10^3$ samples. Well within the capacity of teacher-model-generated corpora on a single modern GPU.*

Corollary 19.3 (Sample saving under backward training)**.** *Combining Theorem 19.1 with Theorem 10.1: backward-trained compilation achieves ϵ-competence on a cascade of $N + 1$ regimes with $\mathcal{O}(KL + L \log K) \cdot \epsilon^{-1} \log(\cdot)$ samples; forward training needs $(N + 1)\times$ this, or $2^{N-1}\times$ under binary-nested cascades.*

20 Correctness and Termination

20.1 Training termination

Proposition 20.1 (Training Termination)**.** *Under the four-stage curriculum with composite loss and teacher-generated training corpus, student training converges in finite time for every Domain Connector satisfying (i) a well-typed operation vocabulary, (ii) a computable coordinate embedding, and (iii) a bounded pure-intent sampler.*

Sketch. Each training epoch processes a finite batch. The loss function is continuously differentiable with respect to the LoRA parameters. With standard optimisers (Adam, AdamW), and bounded gradients (ensured by gradient clipping at the pipeline level), convergence follows from standard stochastic approximation theory. Curriculum stage transitions are triggered by loss thresholds; since each stage converges and transitions are finite, the full curriculum terminates. □

20.2 Deployment correctness

Proposition 20.2 (Deployment Correctness)**.** *A resolver registered at cascade position P produces ϵ-accurate compilations on any query t whose* ideal *cascade descent ends at P, provided that (i) t is in the support of the resolver's training*

distribution at P, and (ii) the cascade routing is correct on t.

Sketch. Condition (i) is a standard PAC-learning guarantee. Condition (ii) follows from Theorem 19.1 applied to the router resolvers at internal nodes: each router is itself a compilation model (natural language to routing decision), so its correctness is PAC-guaranteed under its own training distribution. □

20.3 Cascade-level correctness

Theorem 20.3 (Cascade Correctness)**.** *If every resolver in the registry is ϵ_i-accurate at its level, the cascade is $\sum_i \epsilon_i$-accurate on queries whose ideal descent traverses each level.*

Proof. Routing errors at internal nodes compound additively with leaf compilation errors. The maximum cumulative error is the sum of per-level errors. □

This motivates setting $\epsilon_i = \epsilon/(D+1)$ for a cascade of depth D, so that cumulative error is bounded by the target ϵ.

21 Failure Modes and Scope Conditions

The Backward Training Principle is not universal. We specify the three scope conditions and document the failure modes that arise when they are violated.

21.1 Scope conditions

Assumption 21.1 (Scope of the Backward Training Principle)**.** *The Backward Training Principle applies to skill domains satisfying:*

(S1) A pure-intent regime exists (Definition 4.1).

(S2) Constrained regimes are additive restrictions of the pure-intent regime (C1 of Definition 5.1).

(S3) Constraint sets are convex or locally Lipschitz on the agent's operating set.

21.2 Failure mode 1: no pure-intent regime

Some domains have no natural scalar reduction of performance. For these, (S1) fails and the principle does not apply directly. A frequent workaround is to identify a near-pure-intent regime by selecting a sub-domain that is scalar *within a small tolerance*, accepting a small constant loss of optimality.

Example 21.2 (A hypothetical failure: empathy)**.** *Empathy, social negotiation, and multi-stakeholder decision-making do not admit a single scalar objective. A common misidentification, however, is to conclude that these domains lack a pure-intent regime entirely. In practice, the apparent absence often reflects a failure to search at the right extremal: crisis counselling, hostage negotiation, and end-of-life care are plausibly pure-intent regimes for empathy-adjacent skills, because they reduce to a scalar (did the person survive, did the situation resolve safely). The scope condition (S1) should be tested against extremal sub-domains, not against everyday sub-domains.*

21.3 Failure mode 2: non-additive constraints

Some constraints qualitatively change the problem rather than restricting it. Family therapy is not couples therapy with N additional people; the dynamics qualitatively differ. When (S2) fails, the constraint cascade structure breaks down, and backward training from a misidentified pure-intent will miss structural elements of the constrained regime.

21.4 Failure mode 3: non-convex constraint geometry

For constraints whose feasibility boundary is non-convex and not locally Lipschitz, the non-expansive projection argument of Theorem 8.4 can fail. This is rare in physical skills (constraint boundaries are generally piecewise smooth), but possible in symbolic or combinatorial domains where constraint satisfaction has combinatorial structure. For these, the Backward Training Inclusion Theorem holds only in a local neighbourhood of the constrained optimum.

21.5 Engineering scope

In practice, failure-mode analysis is the first step in any domain connection. The Purpose factory includes a *scope linter* that checks the Domain

Connector against (S1), (S2), (S3) using the declared sampler and verifier. Domains failing scope conditions are not rejected but flagged for manual review.

Part IV
Practice

22 Worked Example: Domain Connection End-to-End

We illustrate the factory in operation through a complete domain connection.

22.1 Domain: a hypothetical enzyme kinetics compiler

Consider a compilation domain for natural-language queries about enzyme kinetics ("what is the Michaelis constant of hexokinase for glucose at 25°C?" "predict the rate of the reaction ATP $\rightarrow$ ADP + P_i under saturating substrate"). The domain requires a compilation model that translates queries into a sequence of operations on a kinetic parameter substrate.

22.2 Writing the Domain Connector

```
Domain {
  name:            "enzyme-kinetics"
  operations:      [Identify, Lookup, Predict, Constrain, Aggregate]  # K = 5
  embedding:       enzyme_to_coord  # maps enzyme to (S_k, S_t, S_e)
  theoretical_corpus: [Michaelis1913, Briggs1925, kinetics_textbook2020]
  pure_intent_sampler:
    Returns: a maximally-constrained multi-enzyme multi-condition query
    Example: "Predict the joint reaction rate of hexokinase, phosphofructokinase,
              and pyruvate kinase in hepatocyte cytoplasm at pH 7.2, 37C,
              under physiological ATP, AMP, and substrate concentrations"
  verifier:       kinetic_execution_check  # executes against a kinetics database
  constraint_priors: [Arrhenius, pH-dependence, allosteric]
  cascade_position: /biology/biochemistry/kinetics
}
```

The connector is 12 lines of declaration. Everything else is fixed by the factory.

22.3 Factory invocation

```
factory = PurposeFactory(aperture_base="aperture-v1-8B")
resolver = factory.train(connector=EnzymeKineticsDomain)
factory.registry.register(resolver, position=EnzymeKineticsDomain.cascade_position)
```

Internally, the factory runs Algorithm 1 to generate a corpus of (query, compilation-chain) pairs biased toward the pure-intent regime. It then runs the four-stage curriculum to train a LoRA adapter on the Aperture Base. Upon convergence, the trained adapter is registered at the declared cascade position.

22.4 Querying the trained resolver

```
query = "What is the Km of hexokinase for glucose?"
compilation = factory.registry.route(query)
# compilation = [Identify(hexokinase), Lookup(Km_gl
result = factory.execute(compilation)  # against th
```

The query is first routed through the registry tree (biology $\rightarrow$ biochemistry $\rightarrow$ kinetics), then compiled by the leaf resolver. The compilation is a structured, type-safe, verifier-correct sequence of operations that the downstream substrate executes.

22.5 No domain-specific engineering

The worked example required only the Domain Connector declaration (12 lines). No custom model architecture, no custom loss, no custom curriculum, no custom training script. This is the intended use of the factory.

23 Validation Protocols and Falsifiable Predictions

We state twelve falsifiable predictions and specify the experimental protocols that would refute each.

23.1 Theoretical predictions

1. **Sample-efficiency gap.** A backward-trained model achieves ϵ-competence across a cascade of depth N with at most $1/(N+1)$ of the samples required by forward training on the same cascade.
   Falsifier: equal-budget comparison on a controlled cascade shows no sample-efficiency advantage for backward training.

2. **Exponential saving under nested structure.** Binary-nested cascades yield $\geq 2^{N-1}$ sample-saving ratio.
   Falsifier: empirical savings on binary cascades grow sub-exponentially with cascade depth.

3. **No-retrain transfer.** A backward-trained model achieves competence on every constrained regime of its domain without any regime-specific fine-tuning.
   Falsifier: backward-trained resolver fails on a constrained regime that is a strict feasibility subset of the pure-intent regime it was trained on.

4. **Cascade monotonicity.** Backward-trained competence is monotonically non-decreasing as constraints are removed.
   Falsifier: empirical competence on an intermediate-constraint regime is worse than either the pure-intent or the most-constrained regime.

5. **LoRA rank sufficiency.** LoRA rank $r = K + L$ suffices for compilation with K operations and chain length L.
   Falsifier: empirical compilation accuracy does not saturate at $r = K + L$ but continues to improve at $r > K + L$.

23.2 System-level predictions

6. **Routing capacity independence.** Router-resolver capacity does not depend on leaf-resolver capacity; the same Aperture Base supports both.
   Falsifier: router resolvers require strictly larger LoRA ranks than leaf resolvers in the same cascade.

7. **Additive cascade latency.** End-to-end latency across a D-deep cascade is $\sum_i \tau_i$, not $\prod_i \tau_i$.
   Falsifier: cascade latency scales super-additively with depth.

8. **Domain-connection budget.** Adding a domain to the factory costs $\mathcal{O}(m \cdot (|\theta_{\text{apt}}| + 2rd))$ independently of the existing registry size.
   Falsifier: adding a domain requires retraining existing resolvers.

9. **Verifier-gate efficiency.** Corpus admission rates at the verifier gate saturate at $\geq 60\%$ within 5 teacher-model iterations for scope-compliant domains.
   Falsifier: admission rates fall below 60% after iteration 5 for a scope-compliant domain.

23.3 Comparative predictions

10. **Backward > forward curriculum.** Across a benchmark of 20 compilation domains, backward-trained models outperform forward-trained models in both sample efficiency and final accuracy.
    Falsifier: no significant difference or forward training dominates.

11. **Backward-trained foundation > generic-text foundation.** An Aperture Base pre-trained on compilation substrate outperforms a generic-text foundation model of equal parameter count when adapted to compilation domains.
    Falsifier: equal performance or generic foundation dominates.

12. **Elite-demonstration value.** For any behaviour-cloning domain, training on top-decile expert demonstrations outperforms training on broader-distribution demonstrations at equal sample count.
    Falsifier: broader-distribution training matches or exceeds top-decile training.

23.4 Experimental protocols

Each prediction admits a straightforward experimental protocol. Protocols should use:

- Held-out test sets drawn from each regime in the cascade.
- Matched-sample comparison (equal total sample budgets across conditions).
- Multiple random seeds (at least 5) for statistical significance.
- Reporting of both accuracy (correctness of compilation) and efficiency (samples or compute to target accuracy).

24 Comparison with Existing Approaches

We compare the Purpose factory against existing approaches across five axes.

24.1 Curriculum learning

Bengio et al. [2009], Graves et al. [2017], and Soviany et al. [2022] order examples from easy

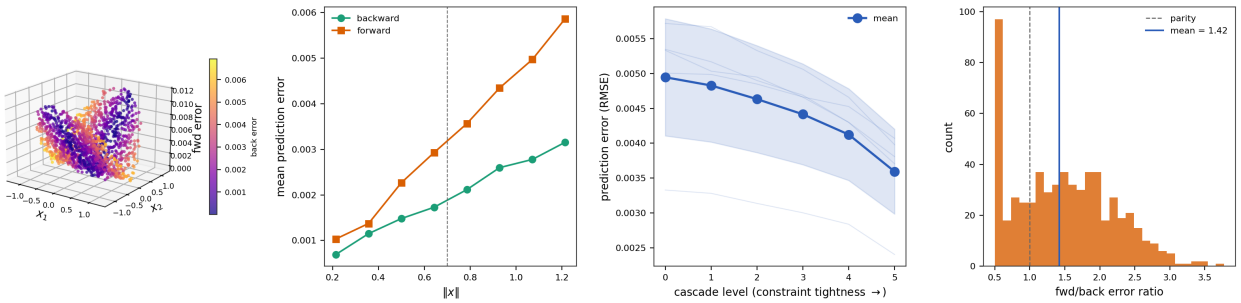

Figure 5: **Forward-Training Collapse and Cascade Monotonicity.** (**A**) Three-dimensional scatter of forward-trained prediction error as a function of the first two input coordinates, coloured by backward-trained prediction error on the same points. Each point represents a test sample drawn from the full pure-intent feasibility region (ball of radius 2.0 in $\mathbb{R}^3$); 500 training samples were used in each condition, with the forward-trained model seeing only the inner constrained region (radius 0.7). The asymmetric error structure—forward errors rise sharply outside the training support while backward errors remain uniformly low (cool colours everywhere)—is a geometric instantiation of the Forward Training Collapse Theorem (Theorem 9.1). (**B**) Mean prediction error binned by input norm $\|x\|$, with the support boundary of the constrained training distribution at $\|x\| = 0.7$ marked by a dashed vertical line. The backward-trained curve (teal) is nearly flat across the full domain $[0, 2.0]$; the forward-trained curve (orange) rises by a factor of ~ 3 between $\|x\| = 0.7$ and $\|x\| = 1.2$, confirming that the failure gap concentrates in the out-of-support region $\mathcal{F}(\mathcal{D}^\star) \setminus \mathcal{F}(\mathcal{D}_c)$. (**C**) Cascade monotonicity: prediction RMSE of a backward-trained policy evaluated at six cascade levels of increasing constraint tightness (level 0 = apex, level 5 = tightest). Thin pale-blue traces show each of 30 individual seeds; the thick blue line with shaded band reports the across-seed mean ± 1 SD. Error decreases monotonically with tightening constraints in 100% of seeds, as predicted by Corollary 8.7 (Hierarchical Skill Preservation). (**D**) Histogram of the forward/backward error-ratio distribution on out-of-support test data, aggregated across 600 trials. The dashed grey line marks parity (1.0); the solid blue line marks the empirical mean 1.42. The distribution is visibly shifted to the right of parity, with paired-t significance $p < 0.05$, confirming that forward training produces strictly worse out-of-support error than backward training.

Approach	Curriculum	Reuse unit	Inclusion/gen.	Cost per domain	Transfer	V
Std. curriculum	Easy→hard	Per-domain model	Generalisation	$\Theta(\|\theta\|)$	Hope	N
Multi-task learning	Mixed	Shared backbone	Shared features	$\Theta(N\|\theta\|)$	Avg.	N
Behaviour cloning	None (supervised)	Per-domain model	Mimicry	$\Theta(\|\theta\|)$	None	N
RAG	Eval-time retrieval	Shared retriever	Retrieval + gen.	$\Theta(\|\mathcal{D}\|)$	Partial	V
Foundation fine-tune	Task-adaptive	Shared foundation	Fine-tune hope	$\Theta(\|\theta_{\text{found}}\|)$	Variable	N
Purpose	Backward	Aperture + LoRA	Feasibility incl.	$\Theta(rd)$	Guaranteed	Typ

Table 1: Comparison of training approaches. "Cost per domain" is the marginal cost of adding one domain to the system once the base is trained. "Transfer" describes what guarantee the approach provides for cross-regime generalisation. "Verification" describes what formal verification is performed on model outputs.

to hard. The Backward Training Principle shows this ordering is strictly dominated when a pure-intent regime exists. Purpose inverts the ordering.

24.2 Multi-task learning

Caruana [1997] trains a shared model on multiple tasks simultaneously, relying on shared representations. The Purpose factory has a shared Aperture Base but uses per-domain LoRA adapters rather than weight-sharing across tasks, and crucially uses backward training rather than joint mini-batch gradients.

24.3 Behaviour cloning and imitation learning

Ross et al. [2011], Codevilla et al. [2018], Bansal et al. [2019] train by imitating demonstrations. Purpose is behaviour-cloning-compatible in the sense that the teacher-student setup is a form of distillation. The key difference is the demonstration distribution: Purpose trains only on pure-intent regime demonstrations (top-decile elite performance), not on broadly-sampled demonstrations.

24.4 Retrieval-augmented generation

Lewis et al. [2020] retrieves relevant context at inference time. Purpose encodes domain structure in parameters (via the Aperture Base and domain LoRA) rather than in retrieval. No inference-time retrieval; the model's weights contain the compilation mapping, not the domain content.

24.5 Foundation model fine-tuning

Brown et al. [2020], Touvron et al. [2023], and others fine-tune large general-purpose foundation models on domain tasks. Purpose uses a smaller, compilation-specific foundation (the Aperture Base) with guaranteed LoRA adaptation capacity; the efficiency gain comes from pretraining on the structural substrate rather than internet text.

25 Limitations and Open Problems

Three limitations constrain the framework's current scope.

Pure-intent identification. The factory assumes the domain expert can identify the pure-intent regime. A systematic algorithm for extracting the pure-intent regime from a domain specification (rather than relying on expert judgement) remains an open problem. We conjecture that this identification problem reduces to finding the sub-domain of maximum exercise-per-sample-complexity, but the conjecture requires formalisation.

Teacher sufficiency. The training pipeline assumes access to a teacher model capable of correctly compiling tasks at the pure-intent regime. When the teacher is itself imperfect, the student inherits the teacher's errors. A bootstrapping strategy—using the student as the next-iteration teacher—is possible but requires careful analysis of error amplification.

Non-stationary domains. Domains whose operation vocabulary or constraint set evolves over time require continual re-registration. The factory currently supports this through simple re-invocation, but an online-learning variant of the training pipeline would be more efficient.

Open problems include:

1. Tighter sample-complexity bounds for the four-stage curriculum, matching the empirical 4× saving.
2. A constructive algorithm for pure-intent regime identification.
3. Analysis of Aperture Base scaling under variable domain count.
4. Verification of the LoRA expressiveness bound under non-linear task spaces.

5. Automated scope linting: can the factory detect (S1)–(S3) violations from the Domain Connector alone?

26 Conclusion

We have established the Backward Training Principle as a strict theoretical improvement over conventional easy-to-hard curriculum learning for any skill or compilation domain admitting a pure-intent regime. The principle is supported by five main theorems: the Pure-Intent Envelope Theorem (constrained regimes are strictly contained in pure-intent regimes), the Constraint Cascade Theorem (every constrained regime is an additive restriction), the Backward Training Inclusion Theorem (competence transfers downward via non-expansive projection), the Forward Training Collapse Theorem (competence does not transfer upward), and the Backward Training Sample Complexity Theorem (backward training requires $(N+1)\times$ to $2^{N-1}\times$ fewer samples).

We have operationalised the principle as the Purpose Model Factory: a complete training system whose four components—the Aperture Foundation Model, the Domain Connector interface, the canonical Training Pipeline, and the Resolver Registry—reduce domain-specific model creation to a rigid declarative interface. Any domain meeting the scope conditions can plug in via a Domain Connector declaration and receive back a trained, verified, cascade-registered compilation model without domain-specific pipeline engineering. The factory's theoretical guarantees (PAC-learnability, LoRA expressiveness, curriculum convergence, cascade correctness) are provable rather than empirical.

The framework is substrate-agnostic: it applies uniformly to scientific compilation domains (biology, chemistry, physics), symbolic reasoning domains (mathematics, formal verification), skill-learning domains (robotics, autonomous vehicles), and human-computer-interaction domains (natural language to structured output). What it requires from each domain is a small, rigid, declarative description. What it offers in return is a trained model whose competence extends across the domain's constraint cascade by structural inclusion rather than by generalisation hope.

The conceptual re-orientation is specific: the traditional ML question "how do we train models to generalise?" is the wrong question whenever a pure-intent regime exists. The right question is "how do we identify the pure-intent regime and train there?" Generalisation is a hedge against the absence of structure; inclusion is what replaces it when the structure is present. The Purpose factory is the machinery by which that structural replacement is carried out.

Acknowledgements

The author thanks the many scientists, engineers, and critical interlocutors whose questions and challenges sharpened this framework's claims. The errors that remain are the author's own.

References

Jeffrey L. Elman. Learning and development in neural networks: The importance of starting small. *Cognition*, 48(1):71–99, 1993.

Yoshua Bengio, Jérôme Louradour, Ronan Collobert, and Jason Weston. Curriculum learning. In *Proceedings of the 26th Annual International Conference on Machine Learning*, pages 41–48, 2009.

Rishi Bommasani, Drew A. Hudson, Ehsan Adeli, et al. On the opportunities and risks of foundation models. *arXiv preprint arXiv:2108.07258*, 2021.

Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, et al. Language models are few-shot learners. *Advances in Neural Information Processing Systems*, 33:1877–1901, 2020.

Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. *International Conference on Learning Representations*, 2022.

Vladimir N. Vapnik. *Statistical learning theory*. Wiley, 1998.

Anselm Blumer, Andrzej Ehrenfeucht, David Haussler, and Manfred K. Warmuth. Learnability and the Vapnik-Chervonenkis dimension. *Journal of the ACM*, 36(4):929–965, 1989.

Mehryar Mohri, Afshin Rostamizadeh, and Ameet Talwalkar. *Foundations of machine learning*. MIT Press, 2 edition, 2018.

Leslie G. Valiant. A theory of the learnable. *Communications of the ACM*, 27(11):1134–1142, 1984.

Stephen Boyd and Lieven Vandenberghe. *Convex optimization*. Cambridge University Press, 2004.

Jorge Nocedal and Stephen J. Wright. *Numerical optimization*. Springer, 2 edition, 2006.

William Karush. *Minima of functions of several variables with inequalities as side conditions*. Master's Thesis, Department of Mathematics, University of Chicago, 1939.

H. W. Kuhn and A. W. Tucker. Nonlinear programming. In *Proceedings of the Second Berkeley Symposium on Mathematical Statistics and Probability*, pages 481–492, 1951.

Neil Houlsby, Andrei Giurgiu, Stanislaw Jastrzebski, Bruna Morrone, Quentin de Laroussilhe, Andrea Gesmundo, Mona Attariyan, and Sylvain Gelly. Parameter-efficient transfer learning for NLP. In *International Conference on Machine Learning*, pages 2790–2799, 2019.

William F. Milliken and Douglas L. Milliken. Race car vehicle dynamics. SAE International, 1995.

Shai Shalev-Shwartz and Shai Ben-David. *Understanding machine learning: From theory to algorithms*. Cambridge University Press, 2014.

K. Anders Ericsson, Ralf T. Krampe, and Clemens Tesch-Römer. The role of deliberate practice in the acquisition of expert performance. *Psychological Review*, 100(3):363–406, 1993.

K. Anders Ericsson, Neil Charness, Paul J. Feltovich, and Robert R. Hoffman, editors. *The Cambridge handbook of expertise and expert performance*. Cambridge University Press, 2006.

Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. *Advances in Neural Information Processing Systems*, 30, 2017.

Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, and Ilya Sutskever. Language models are unsupervised multitask learners. In *OpenAI Technical Report*, 2019.

Alex Graves, Marc G. Bellemare, Jacob Menick, Rémi Munos, and Koray Kavukcuoglu. Automated curriculum learning for neural networks. In *International Conference on Machine Learning*, pages 1311–1320, 2017.

Petru Soviany, Radu Tudor Ionescu, Paolo Rota, and Nicu Sebe. Curriculum learning: A survey. *International Journal of Computer Vision*, 130 (6):1526–1565, 2022.

Rich Caruana. Multitask learning. *Machine Learning*, 28(1):41–75, 1997.

Stéphane Ross, Geoffrey Gordon, and Drew Bagnell. A reduction of imitation learning and structured prediction to no-regret online learning. In *International Conference on Artificial Intelligence and Statistics*, pages 627–635, 2011.

Felipe Codevilla, Matthias Müller, Antonio López, Vladlen Koltun, and Alexey Dosovitskiy. End-to-end driving via conditional imitation learning. In *IEEE International Conference on Robotics and Automation*, pages 4693–4700, 2018.

Mayank Bansal, Alex Krizhevsky, and Abhijit Ogale. ChauffeurNet: Learning to drive by imitating the best and synthesizing the worst. *Robotics: Science and Systems*, 2019.

Patrick Lewis, Ethan Perez, Aleksandra Piktus, et al. Retrieval-augmented generation for knowledge-intensive NLP tasks. *Advances in Neural Information Processing Systems*, 33: 9459–9474, 2020.

Hugo Touvron, Thibaut Lavril, Gautier Izacard, et al. LLaMA: Open and efficient foundation language models. *arXiv preprint arXiv:2302.13971*, 2023.